iNeuOS 工业互联网操作系统

应用手册

2021 年 05 月 29 日

目录

1. 设备模型（数据采集）.....4
1.1 Modbus 协议.....4
1.1.1 打开设备模型应用.....4
1.1.2 增加设备.....4
1.1.3 配置服务实例.....5
1.1.4 配置设备驱动信息.....7
1.1.5 配置采集数据点.....8
1.1.6 重新加载配置.....10
1.1.7 验证读取数据.....11
1.2 PLC 协议.....13
1.2.1 打开设备模型.....13
1.2.2 配置服务实例.....14
1.2.3 配置连接 PLC 通讯信息.....14
1.2.4 配置读取 PLC 数据点.....15
1.2.5 重新加载配置.....15
1.2.6 新建应用视图.....16
1.2.7 拖动和配置应用组件.....16
1.2.8 保存及应用.....17
2. 视图建模（数据组态）.....18
2.1 概述.....18
2.2 编辑区域.....19
2.3 操作流程及应用.....19
2.4 3500 多个 SVG 矢量图元.....23
2.5 多方案多级别脚本预警配置.....24
2.6 单击和右键菜单响应事件.....25
2.7 上传图元和自定义绘制图形.....26
2.8 丰富的图表数据显示组件.....26
2.9 低延时实时视频接入.....27

2.10 控制命令优化级下发.....28
2.11 开放式的二次开发接口.....28
3. 资产模型（机器学习）.....29
3.1 部署配置.....29
3.2 操作过程.....29
4. 联动控制.....35
4.1 前言.....35
4.2 内置函数.....36
4.2.1 ctrl.....36
4.2.2 save.....36
4.2.3 getvalue.....36
4.2.4 getpara.....37
4.2.5 delay.....37
4.2.6 log.....37
4.2.7 start.....37
4.2.8 stop.....37
4.2.9 exit.....38
4.3 自定义封装函数.....38
4.3.1 tryParseDouble.....38
4.3.2 awaitCtrl.....39
4.4 使用介绍.....40

1. 设备模型（数据采集）

1.1 Modbus 协议

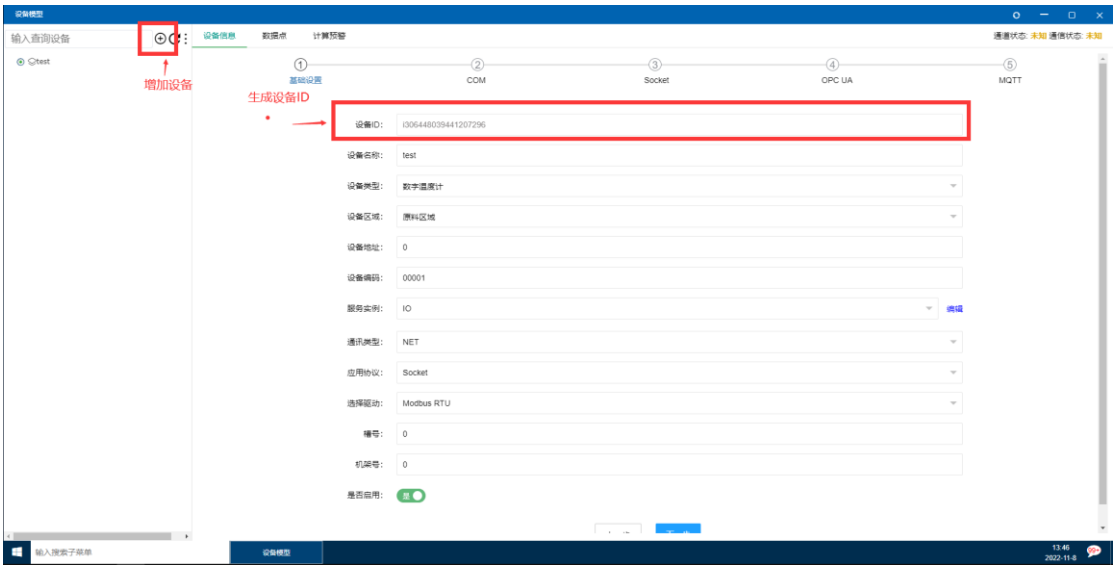
1.1.1 打开设备模型应用

打开系统桌面的【设备模型】，如下图：



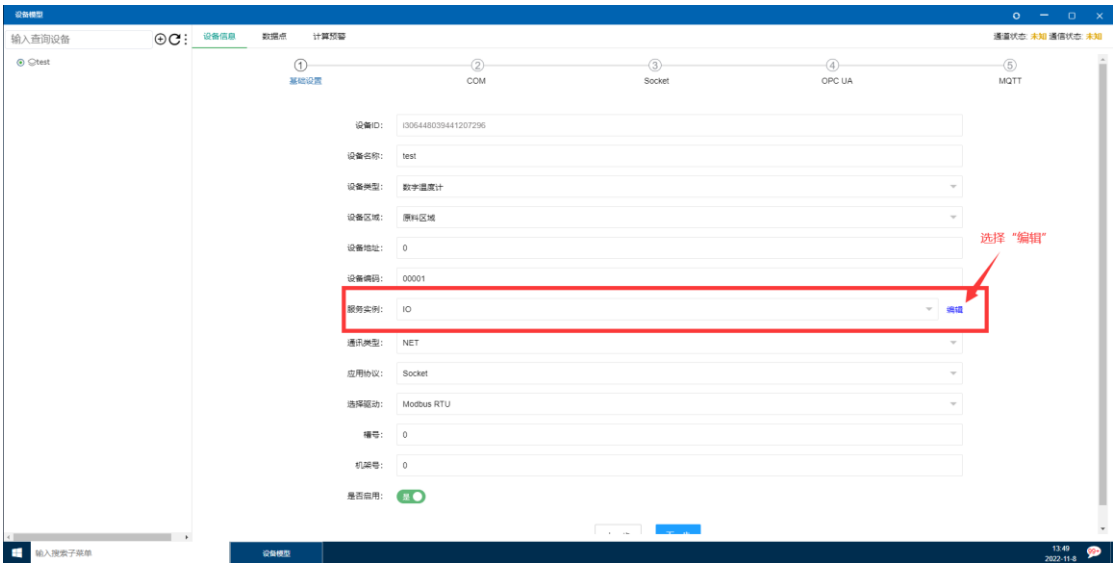
1.1.2 增加设备

单击左上面的⊕增加设备，并且在右边会生成“设备 ID”。如下图：

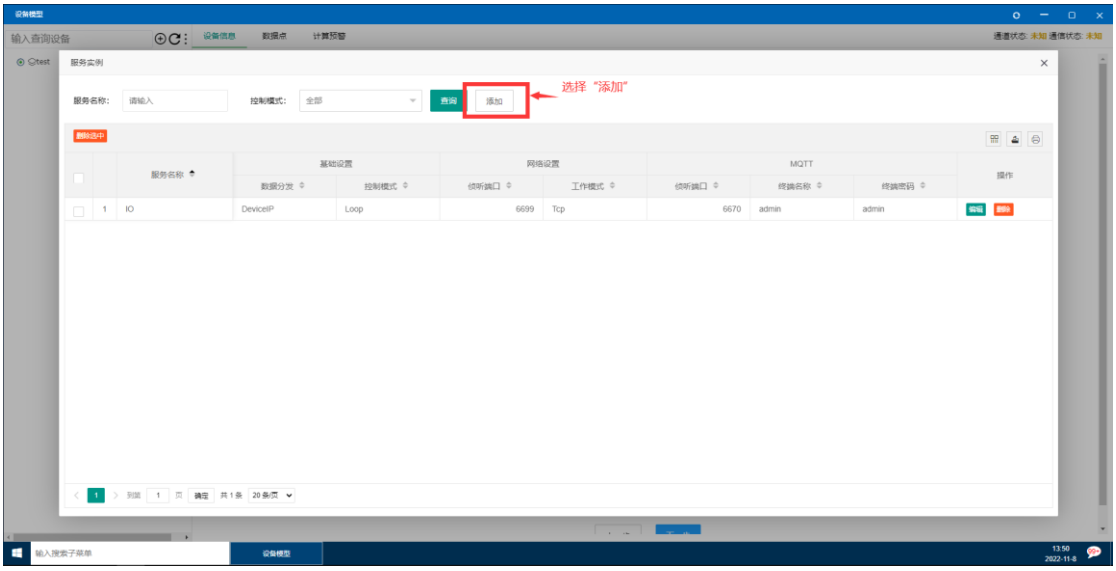


1.1.3 配置服务实例

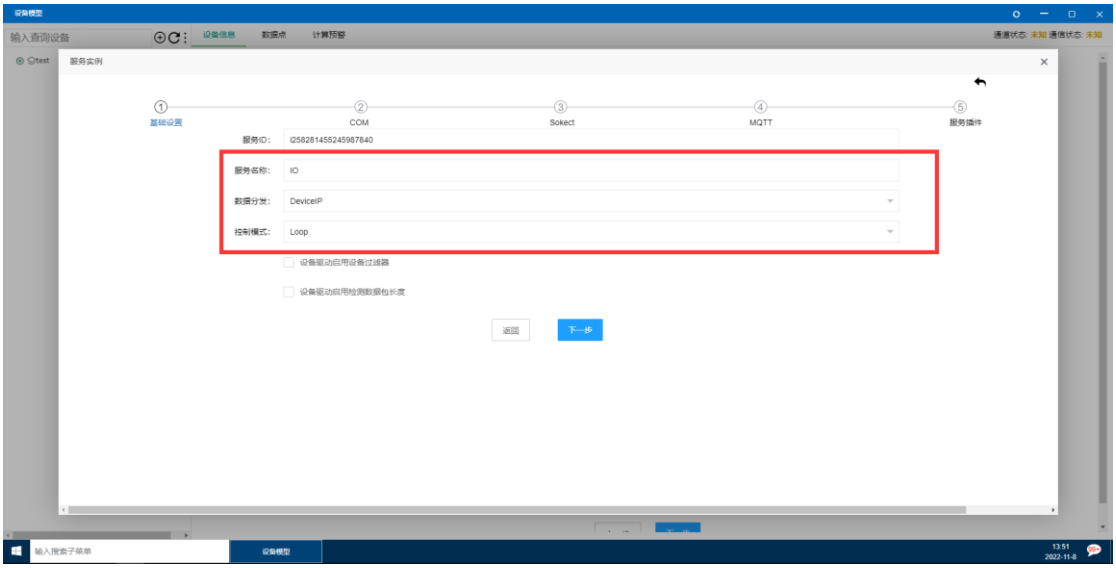
服务实例是增加设备通讯效率的一种方式，每个设备可以配置不同的服务实例。在【服务实例】右边选择“编辑”按钮。如下图：



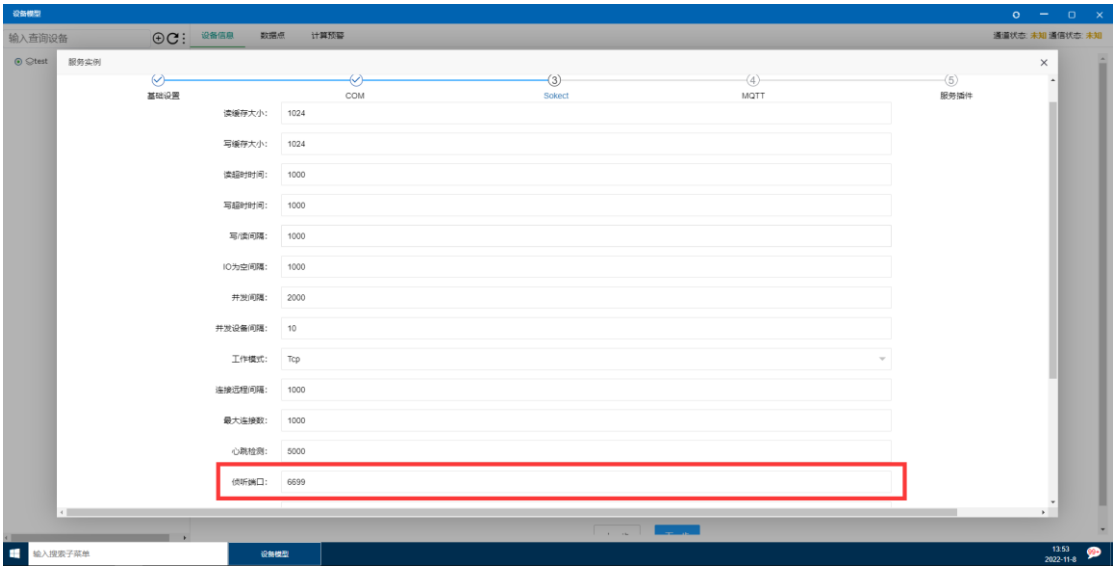
添加服务实例，如下图：

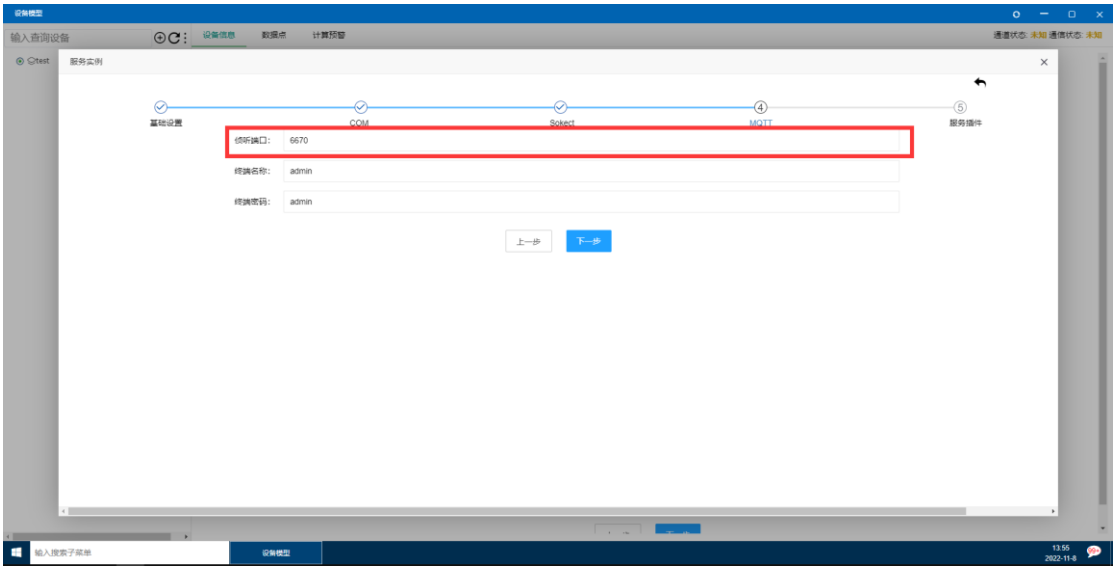


配置服务实例，填写服务名称（自定义）、数据分发（一般选择 DeviceIP，但是要求 Modbus 终端设备的 IP 不相同）、控制模型（选择 Loop 轮询模式）。如下图：

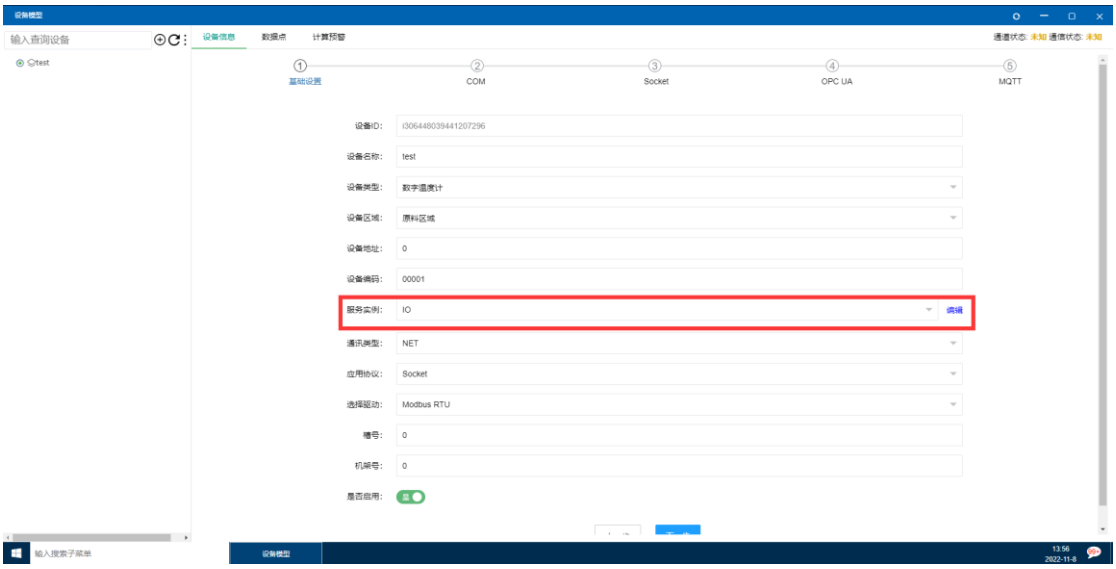


每个服务实例的 Socket 和 MQTT 侦听端口不能相同。如下图：



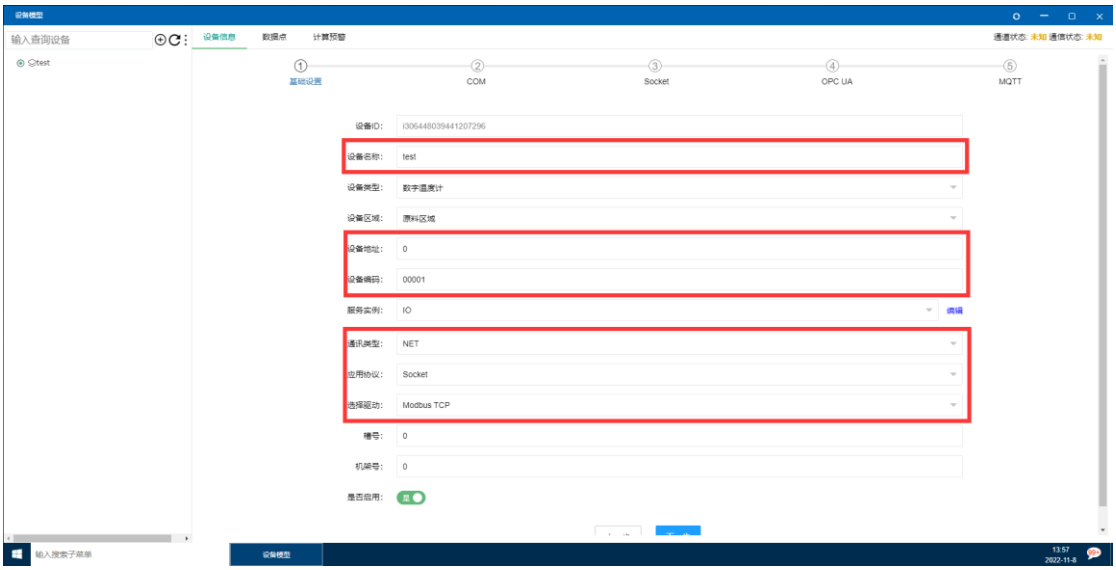


选择配置好的服务实例，如下图：

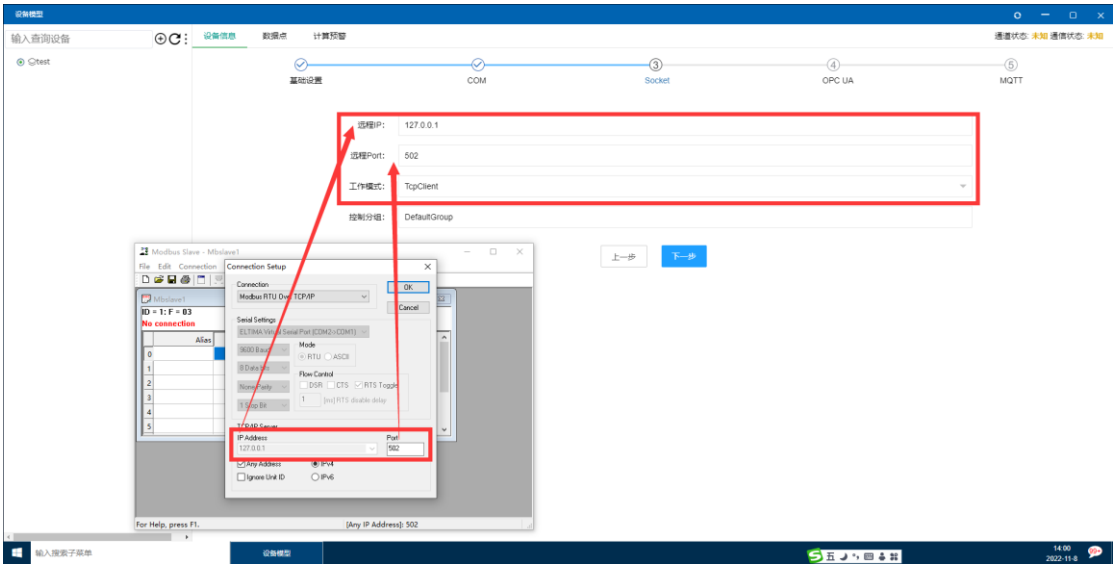


1.1.4 配置设备驱动信息

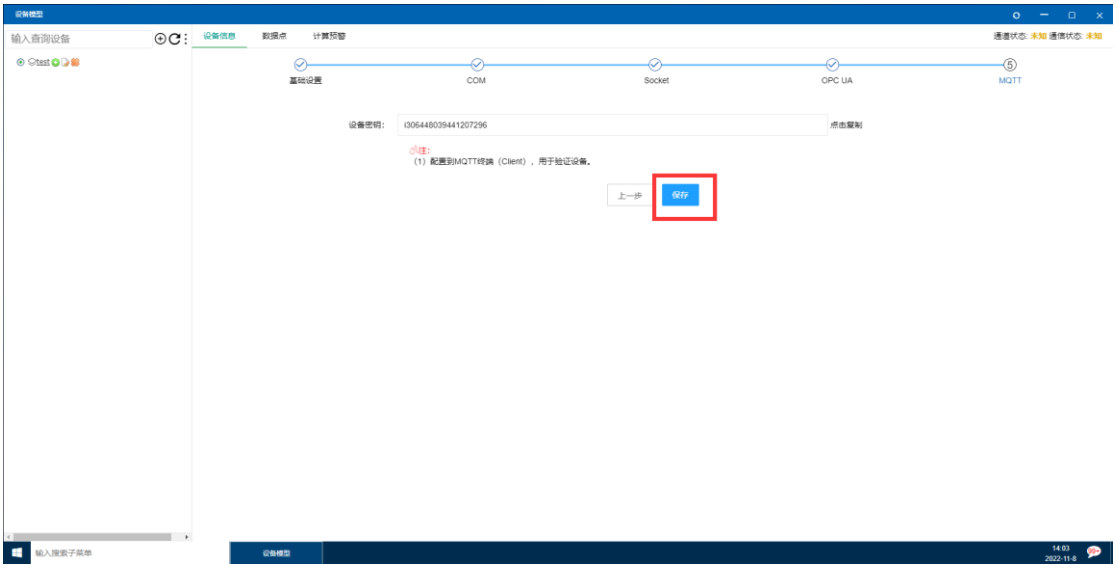
配置设备驱动信息包括：设备名称、设备地址（只是标识）、设备编码（数据分发为 DeviceCode 的时候使用）、通讯类型、应用协议和选择驱动。如下图：



选择【Socket】选项卡，配置连接 Modbus 终端的远程 IP、远程 Port 和工作模式（选择 TcpClient）。如下图：

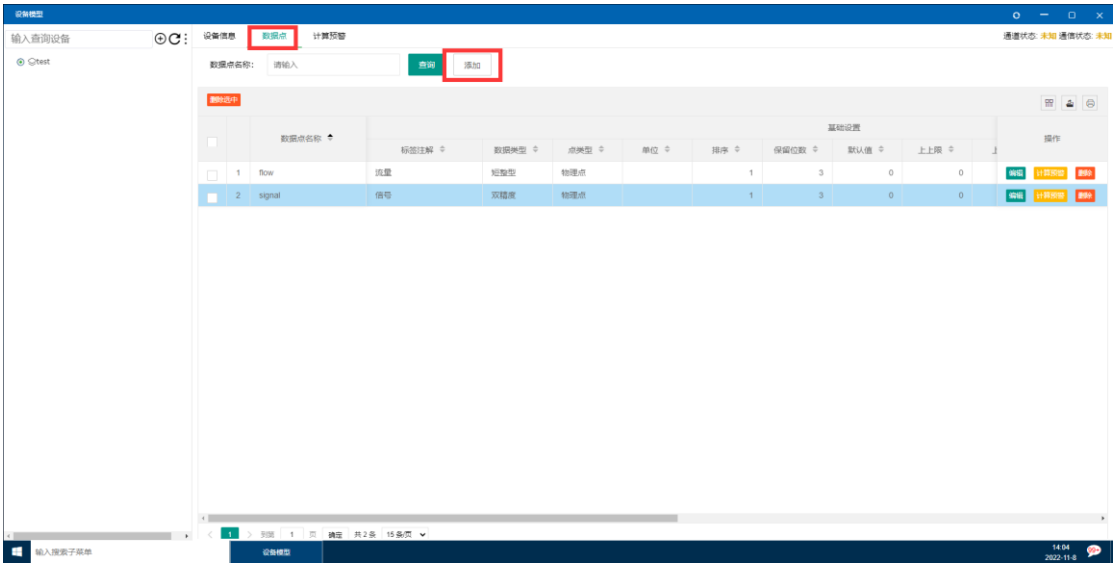


配置完【设备信息】，一直【下一步】，选择【保存】。如下图：

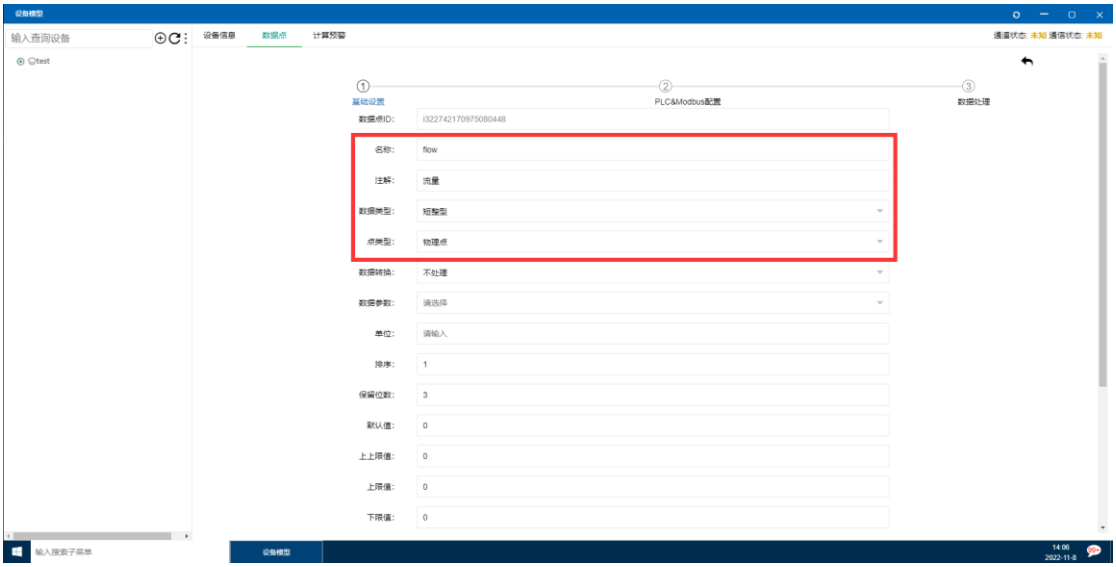


1.1.5 配置采集数据点

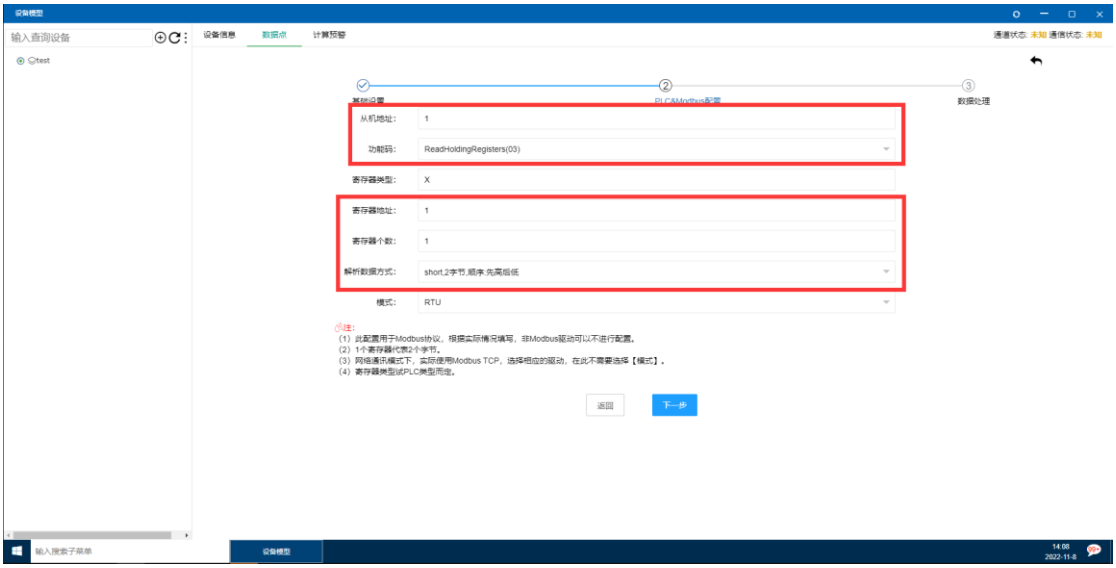
选择设备的【数据点】，单击【添加】。如下图：



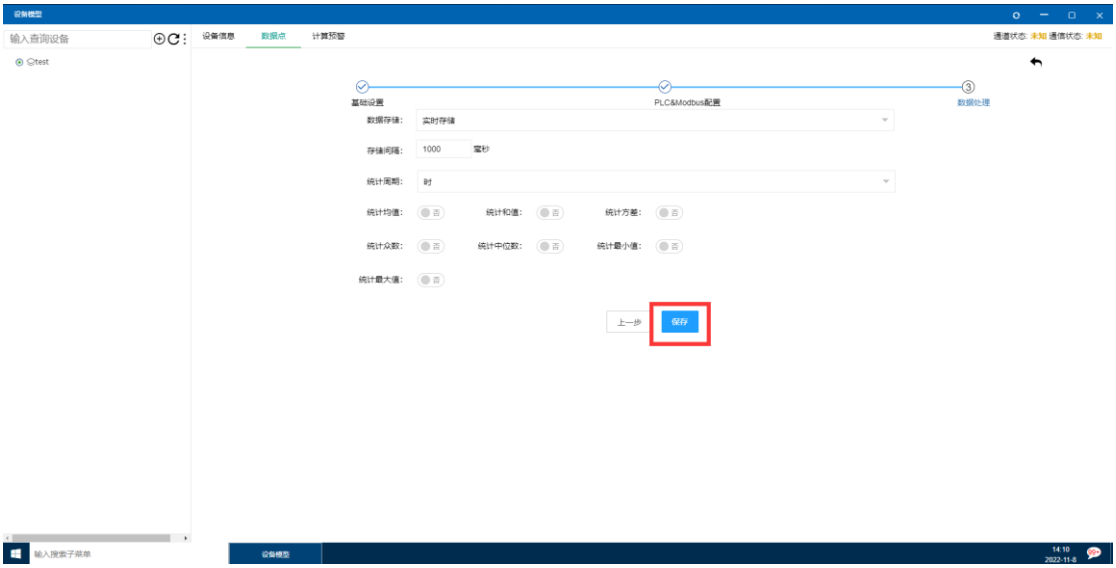
在【基础配置】选卡中，配置数据点的名称、注解、数据类型和点类型（物理点）。如下图：



在【PLC&Modbus 配置】选卡中，按 Modbus 设备终端的实际情况配置从机地址、功能码、寄存器地址、寄存器个数和解析数据方式。如下图：



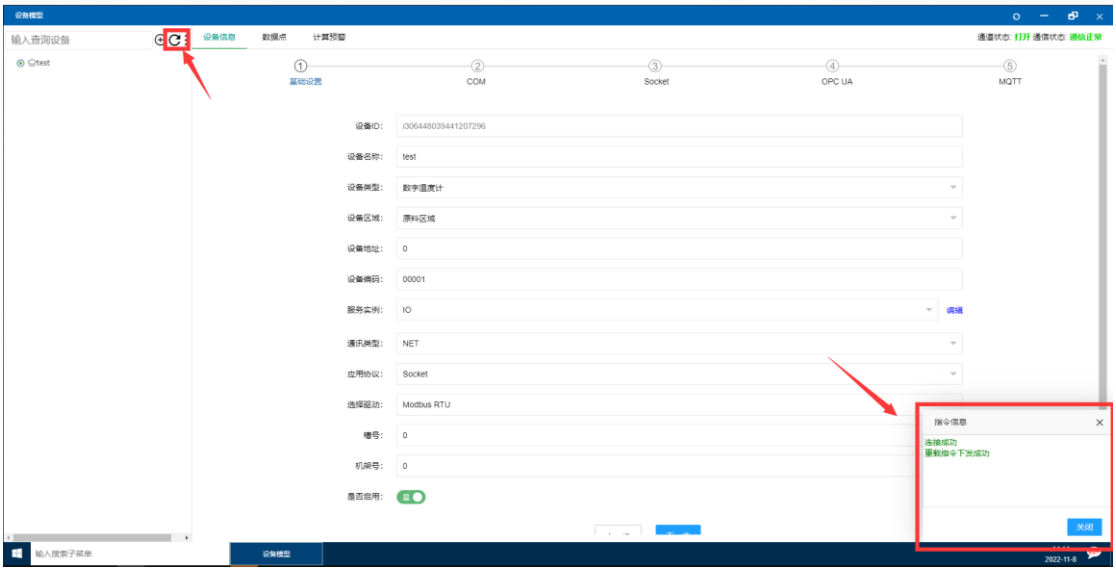
一直单击【下一步】，选择【保存】。如下图：



一个采集数据点创建成功。

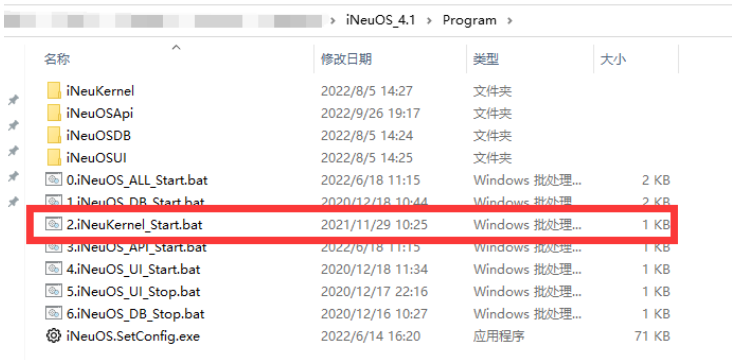
1.1.6 重新加载配置

选择左上面的↻按钮，重新加载刚才的配置信息。如下图：



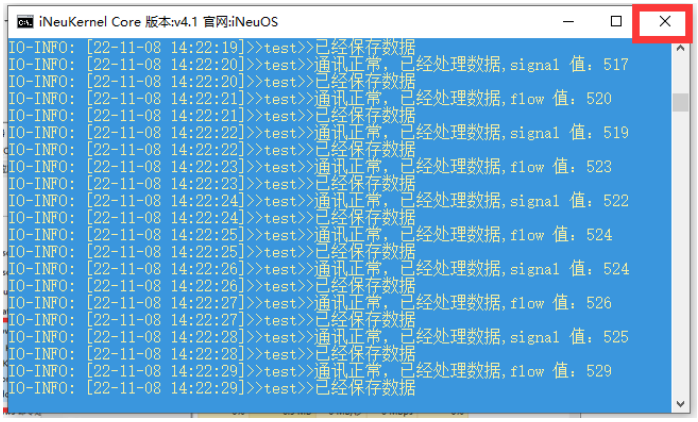
1.1.7 验证读取数据

核实后台服务是否运行，在“iNeuOS_4.1\Program\2.iNeuKernel_Start.bat”目录下。如下图：

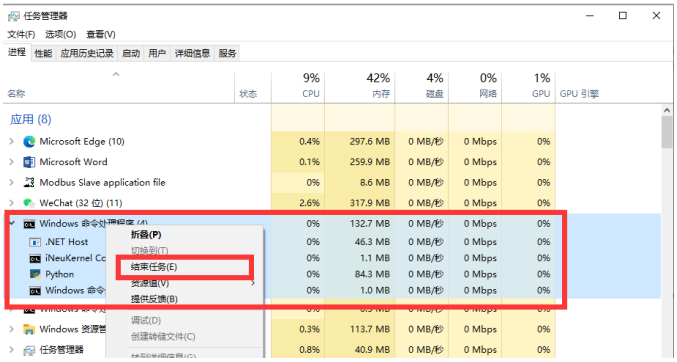


也可以手动关闭或是在任务管理器关闭后台服务进程，然后手动启动 iNeuKernel_Start.bat 服务。

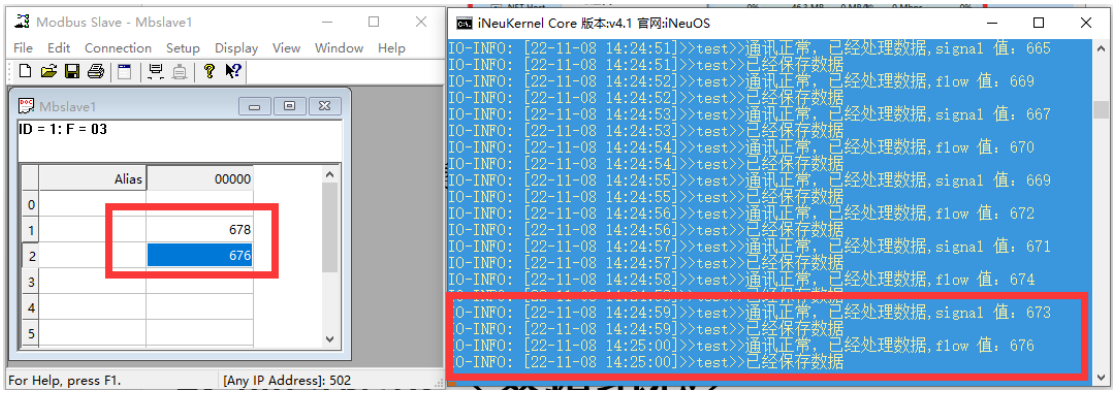
手动关闭 iNeuKernel Core 进程。如下图：



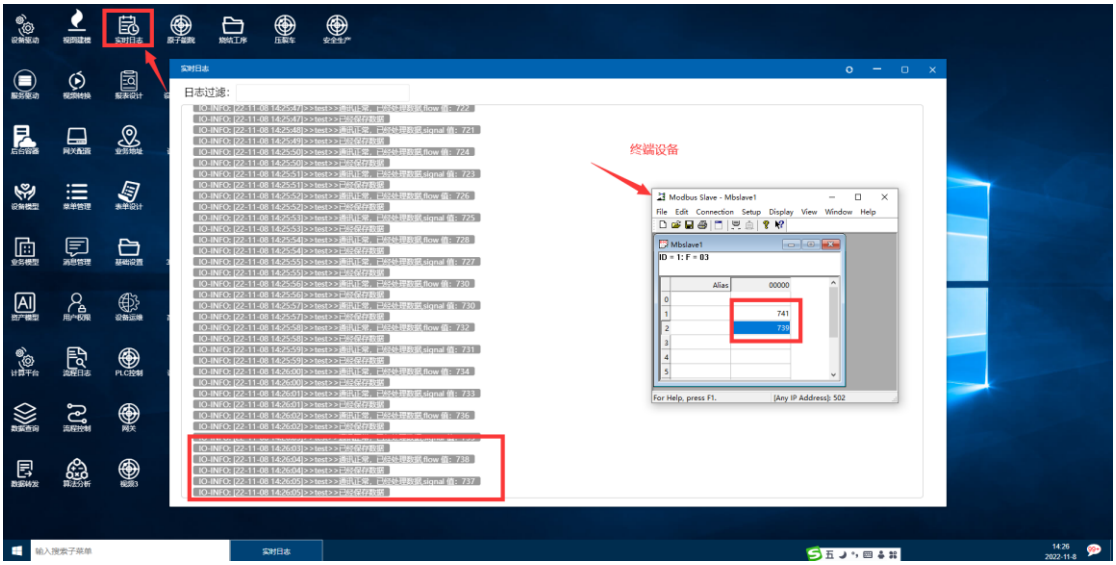
在任务管理器关闭后台进程，右键选择带“iNeuKernel Core”名称的进程，选择【结束任务】。如下图：



正常读取采集点的数据，如下图：



也可以在系统桌面打开【实时日志】查看实时通讯日志。如下图：



完成读取 Modbus 终端设备数据完成，特殊情况按实际场景配置。

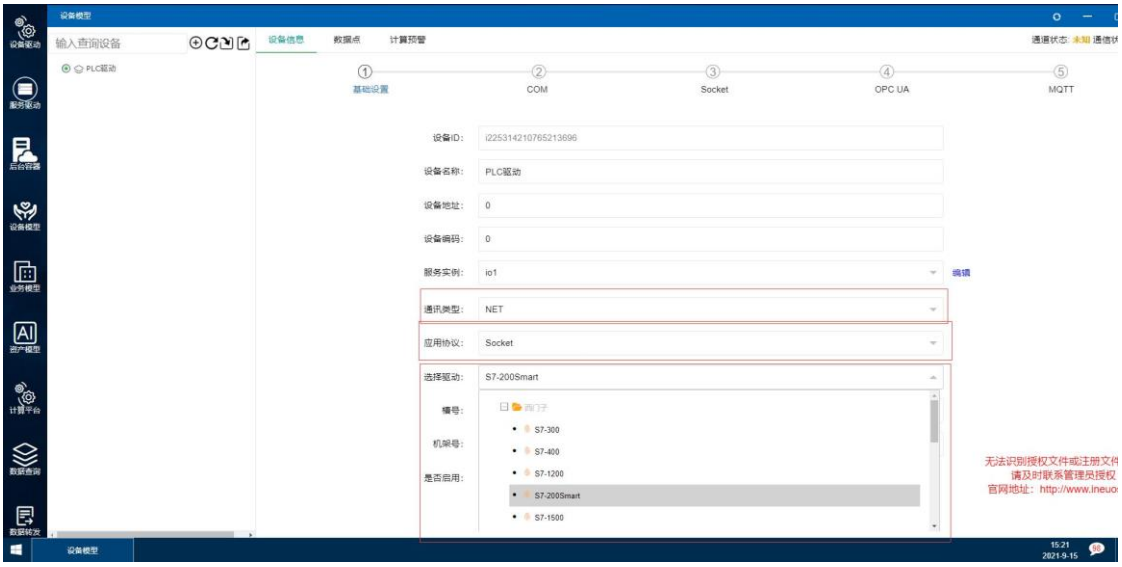
1.2 PLC 协议

1.2.1 打开设备模型

打开桌面的【设备模型】，如下图：

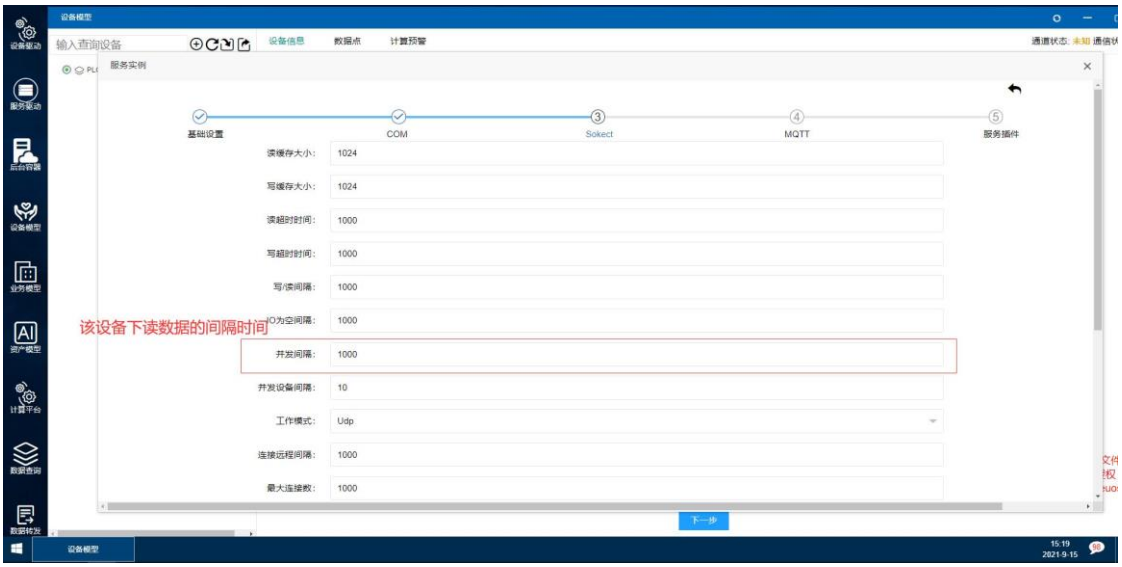


单击左上边的“+”号，新增加一个设备驱动，在“基础设置”中设置设备名称、通讯类型、应用协议、选择驱动等。如下图：



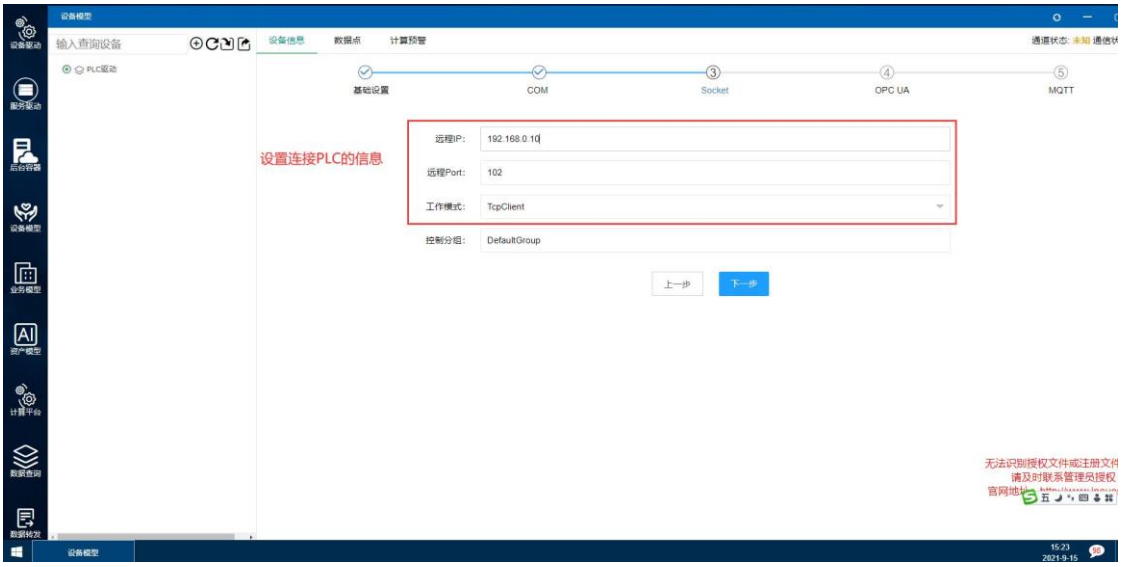
1.2.2 配置服务实例

在“服务实例”选项右边选择“编辑”按钮，新增加一个服务实例，并设置 Socket 的“并发间隔”参数，这个参数主要影响读取 PLC 数据点的周期，参数值越小读数据的效率越高，但是要根据现场的情况设置。如下图：



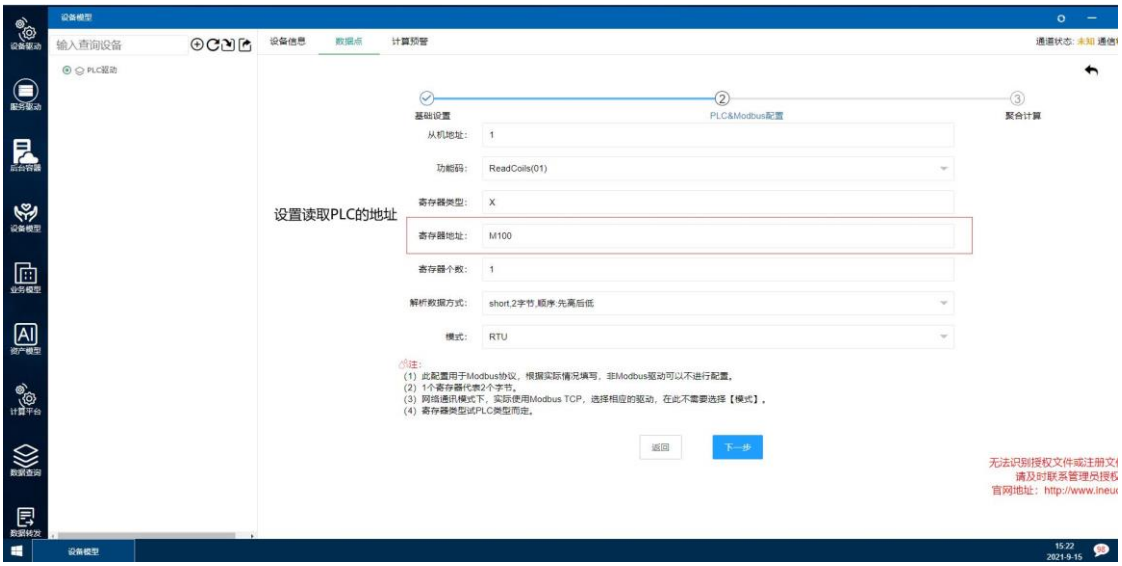
1.2.3 配置连接 PLC 通讯信息

在“设备信息”的“Socket”的选项卡中设置连接 PLC 的 IP 地址、端口号和工作模式等。如下图：



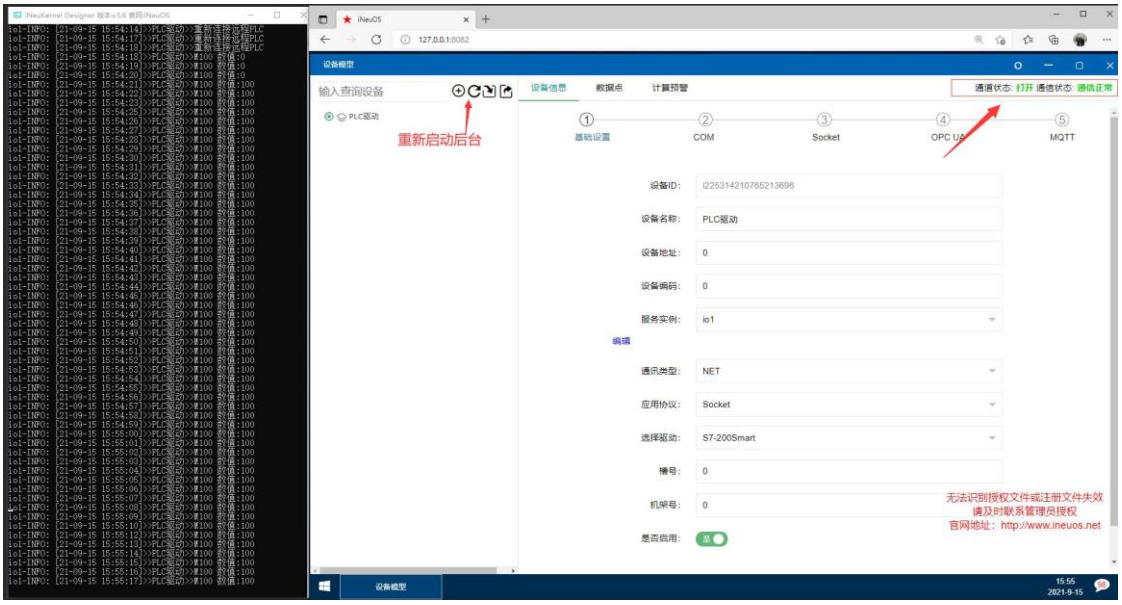
1.2.4 配置读取 PLC 数据点

在设备的“数据点”选项卡里选择“PLC&Modbus 配置”中设置 PLC 的“寄存器地址”，也就是 PLC 的地址。如下图：



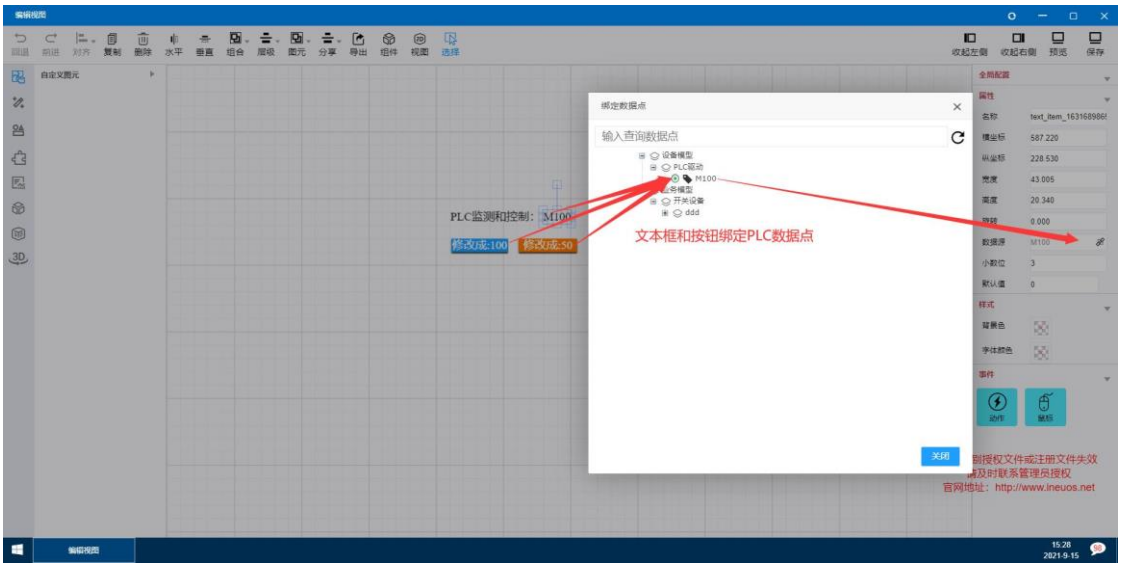
1.2.5 重新加载配置

配置好设备驱动后，选择左上角转圈的按钮，重新启动后台服务。会在右上角显示当前 PLC 设备的通道状态和通信状态。如下图：



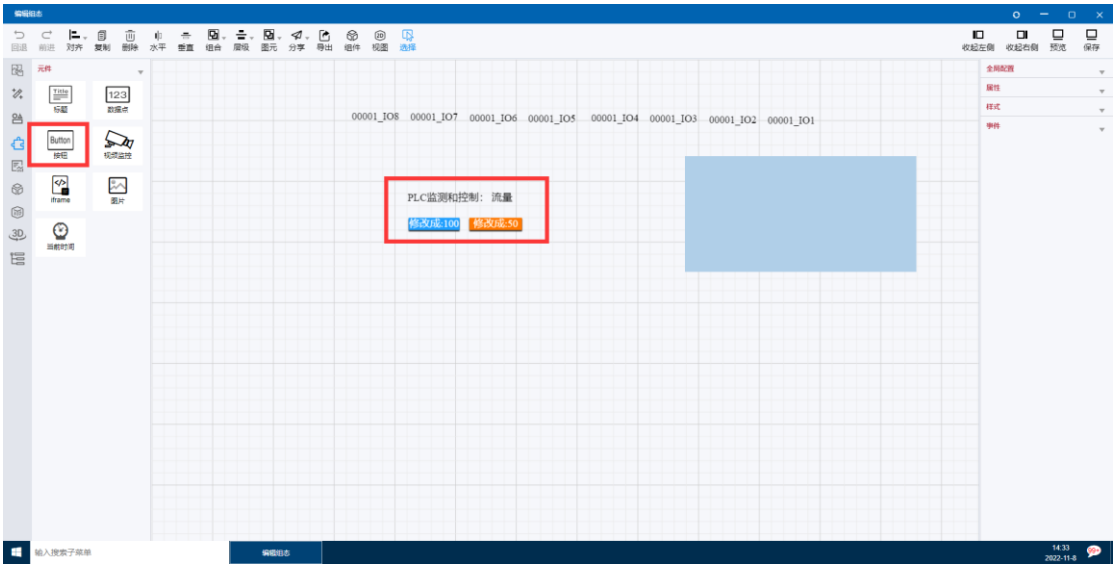
1.2.6 新建应用视图

在 iNeuOS 系统的桌面打开【视图建模】的组态功能，拖到画上标题框、文本框和按钮等组件，并且绑定数据点。如下图：

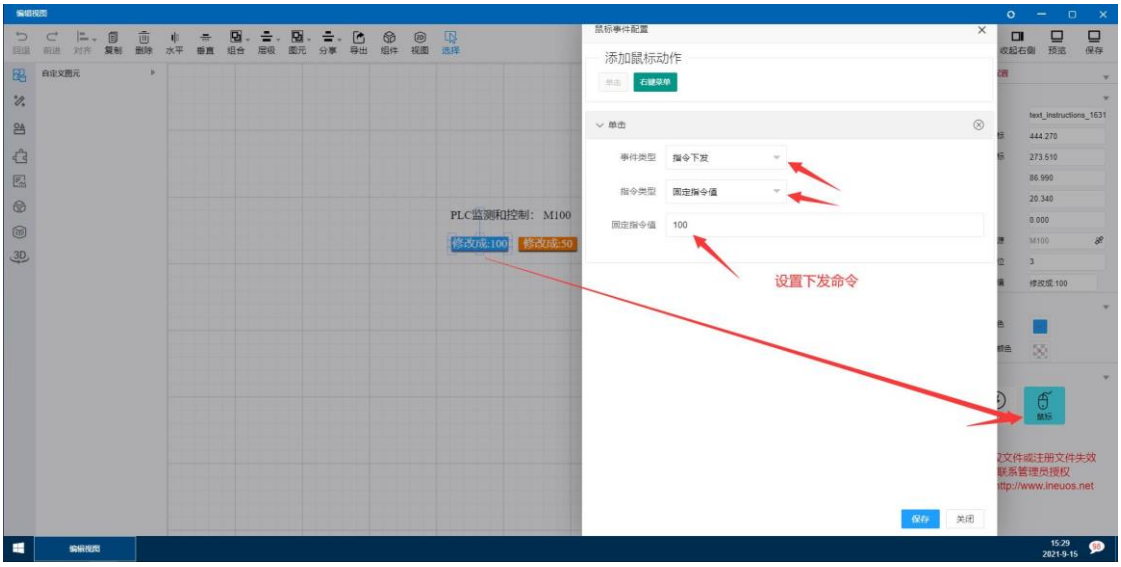


1.2.7 拖动和配置应用组件

选择按钮组件，设置标题。如下图：

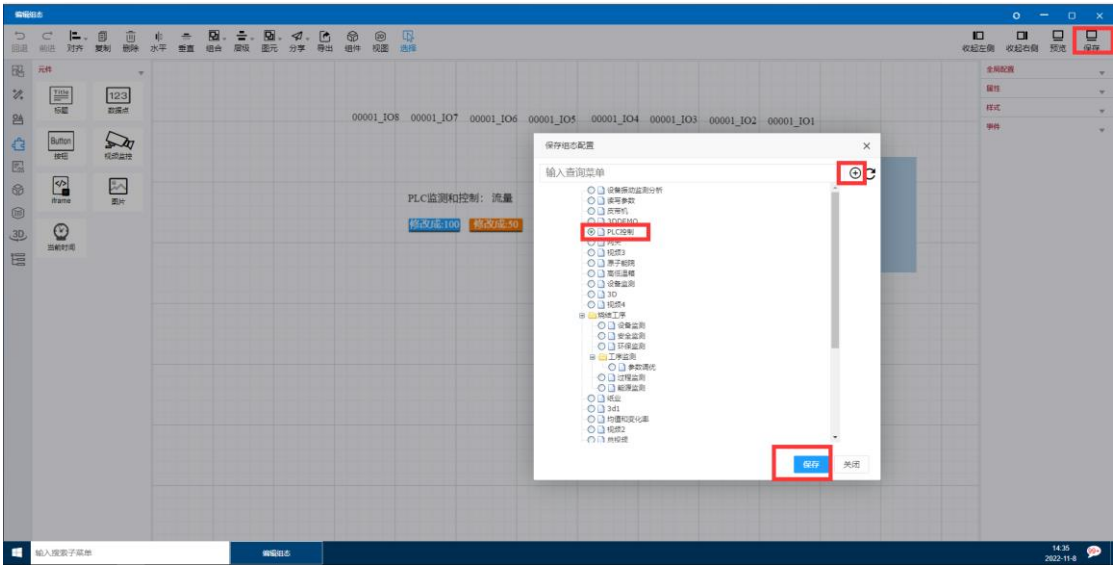


设置按钮鼠标事件，设置事件类型、指令类型和修改 PLC 数据点的值。如下图：

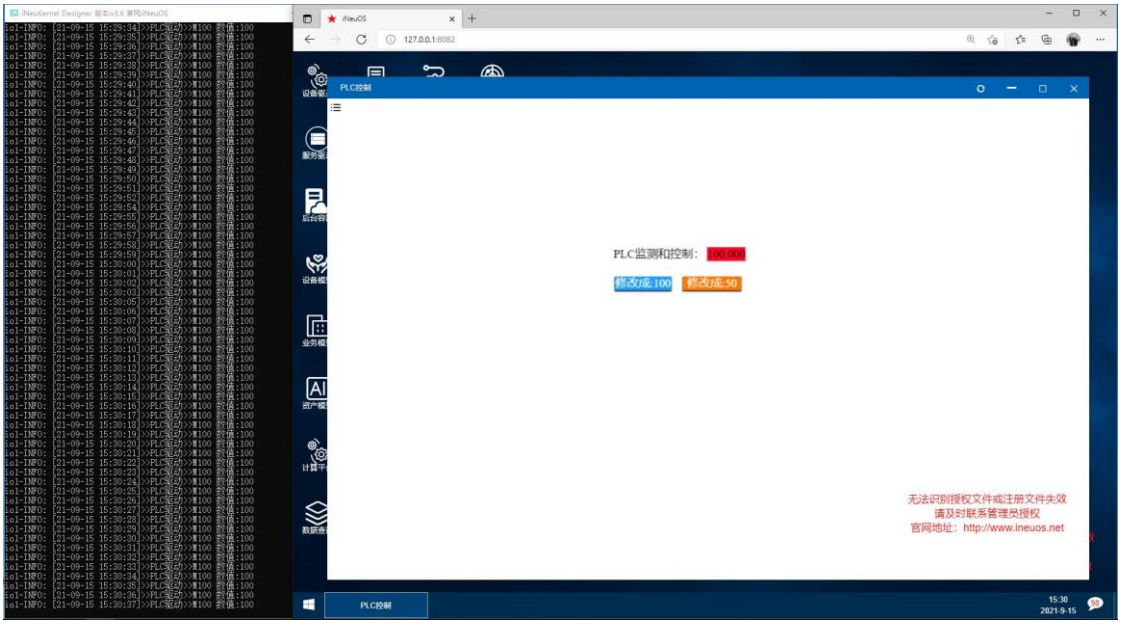


1.2.8 保存及应用

编辑好之后，保存当前组态应用。如下图：



在桌面会有一个【PLC 控制】对应的应用，双击打开。效果如下图：



读取 PLC 数据和控制配置应用完成，有特殊情况，请按实际应用场景配置。

2. 视图建模（数据组态）

2.1 概述

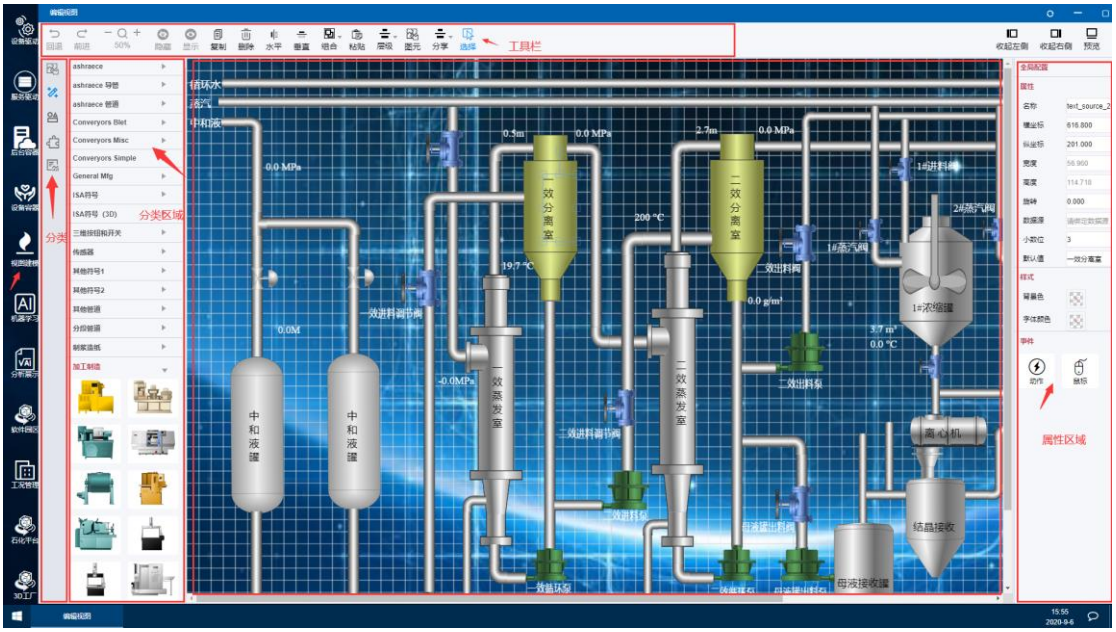
iNeuOS 操作系统中的 iNeuView 视图建模平台，对 iNeuKernel 接入的设备或传感器数据进行视图建模、组态监测和实时预警等。

主要特性包括：3500 多 SVG 矢量图元、多方案多级别脚本预警配置、单击和右键菜单响应事件、上传图元和自定义绘制图形、丰富的图表数据显示组件、低延时实时视频接入、控制命令优化级下发和开放式的二次开发接口等。

iNeuView 组态与物联网框架 iNeuKernel 结合使用，完成硬件设备及其数据的即接、即用、即显示，完全的 B/S（浏览器）应用框架，像应用 Windows 程序一样使用 iNeuView 进行视图建模。

2.2 编辑区域

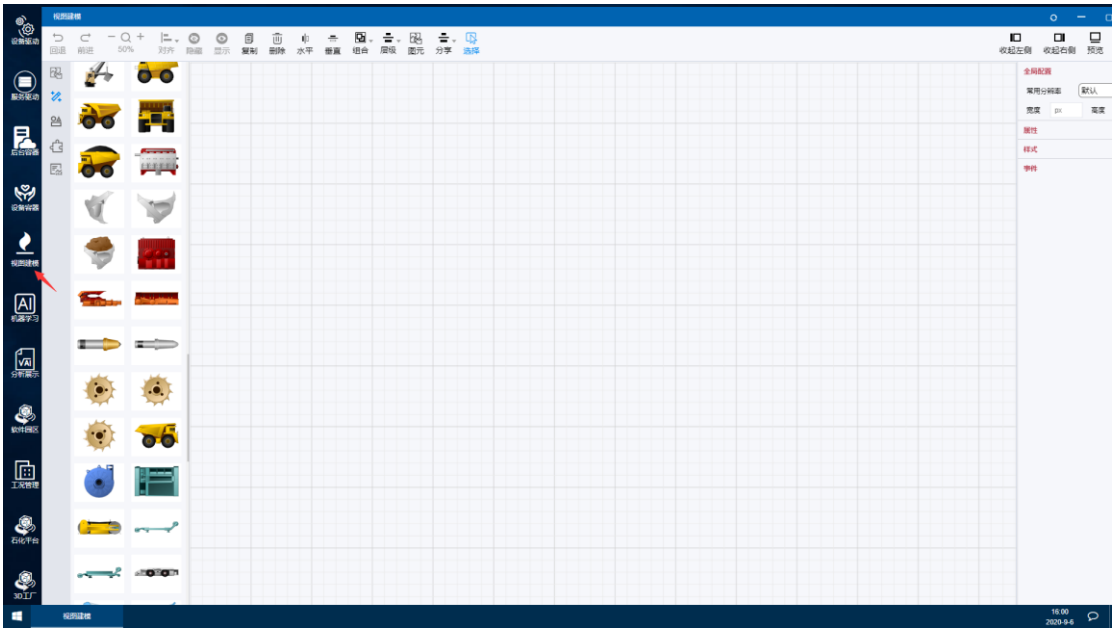
编辑区域分为：分类、分类区域、编辑区域、工具栏和属性区域等。如下图：



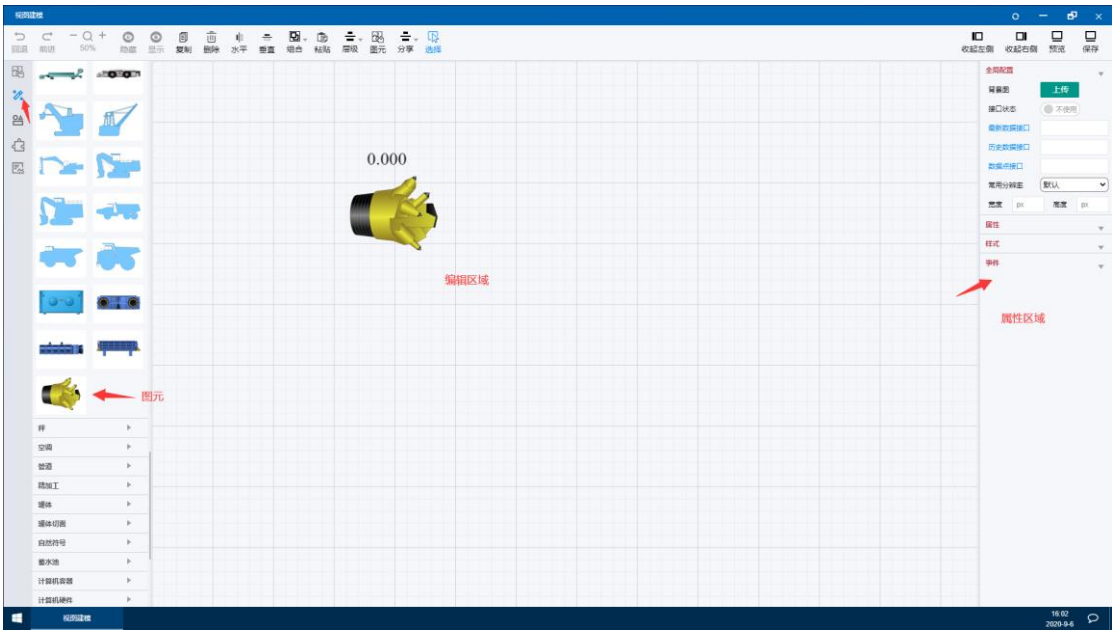
2.3 操作流程及应用

1、打开 iNeuOS 演示地址，输入用户名称和密码。

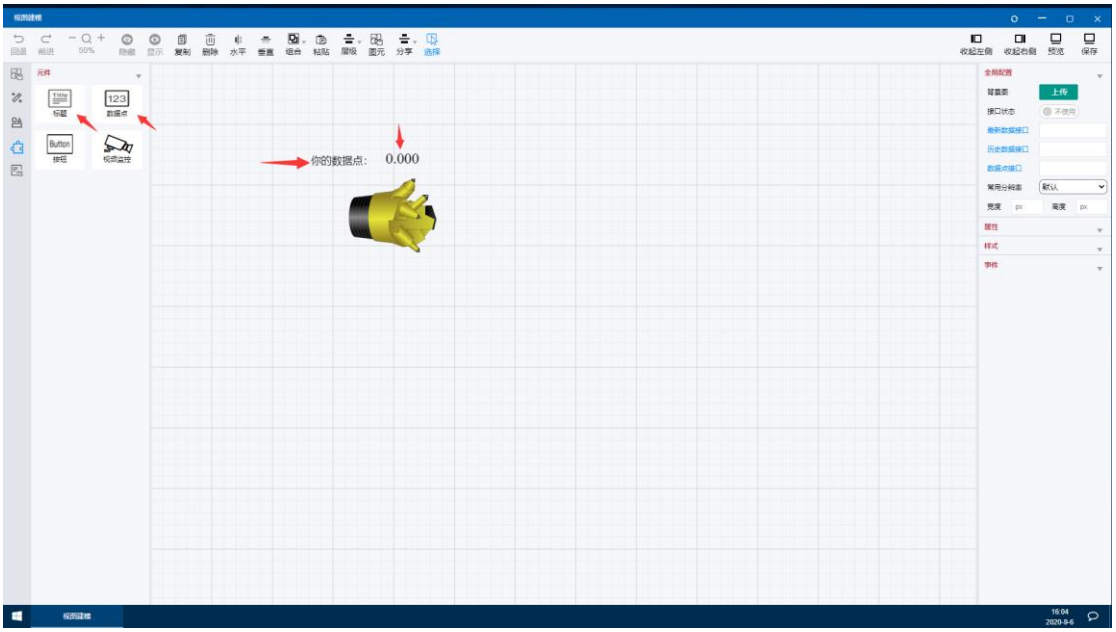
2、双击【视图建模】平台，如下图：



3、拖拽左侧的图元信息，到编辑区域，如下图：



4、标注文字，把【标题】框拖拽到相应的位置，并且编辑相应的信息，如下图：



5、绑定数据点，把【数据点】框拖拽到相应的位置，单击【数据源】绑定数据点，如下图：

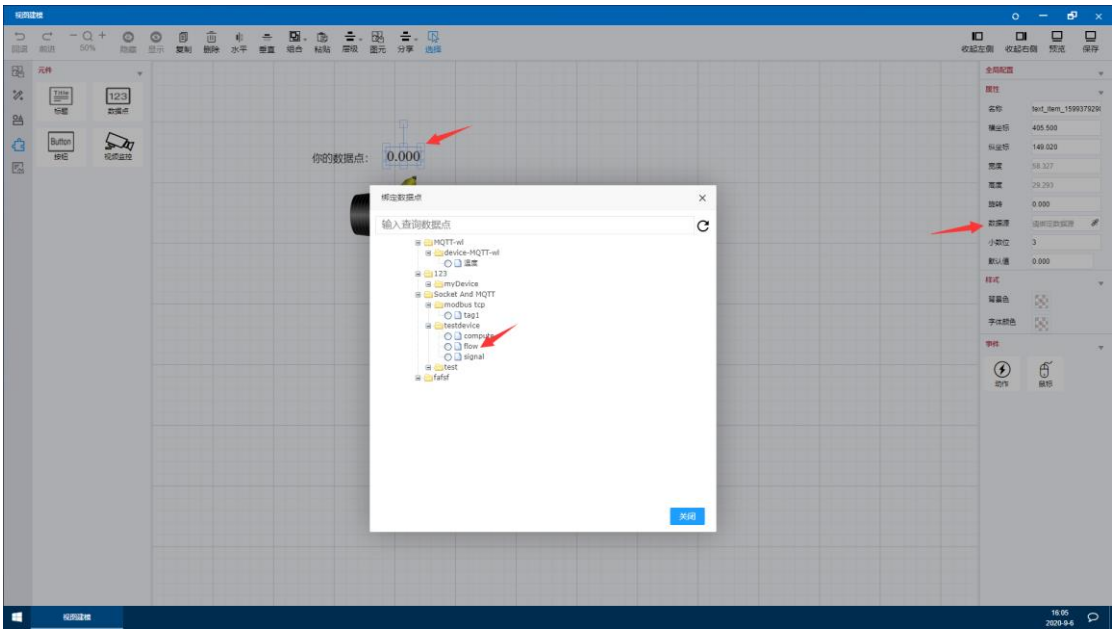


图 绑定数据点

6、应用发布，可以把编辑好的界面发布到 iNeuOS 桌面，单击【保存】，新建要保存的应用的名称，如下图：

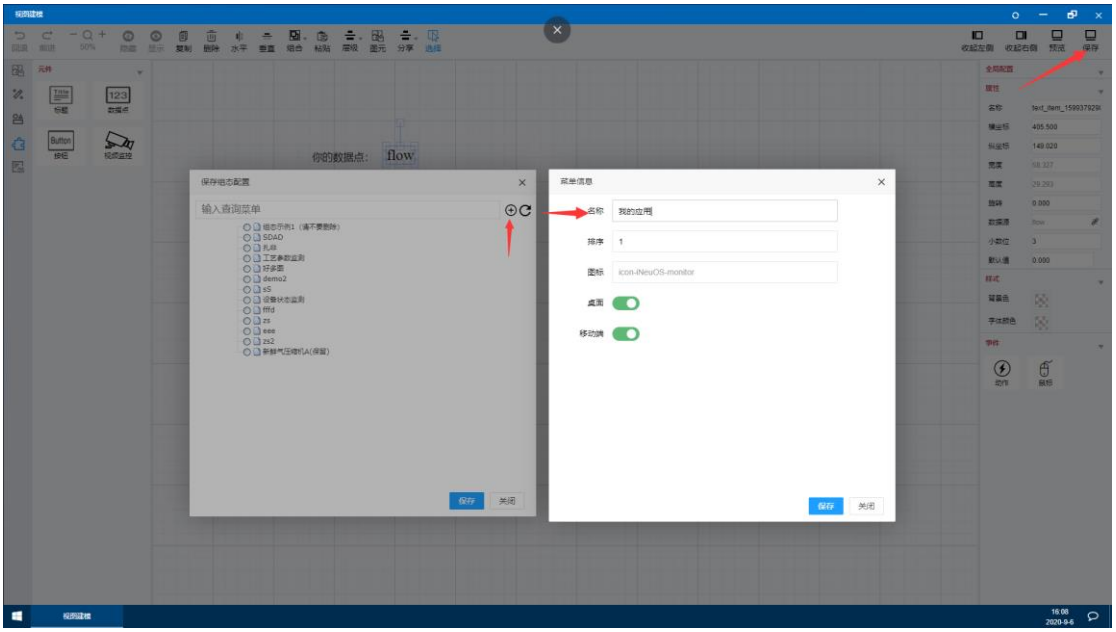


图 应用保存

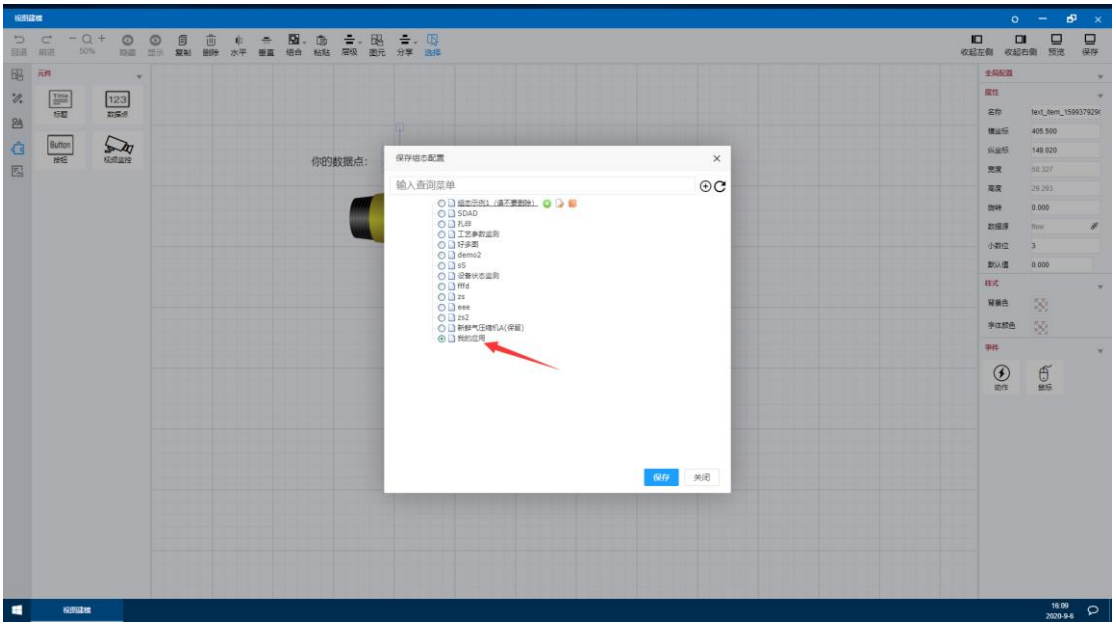
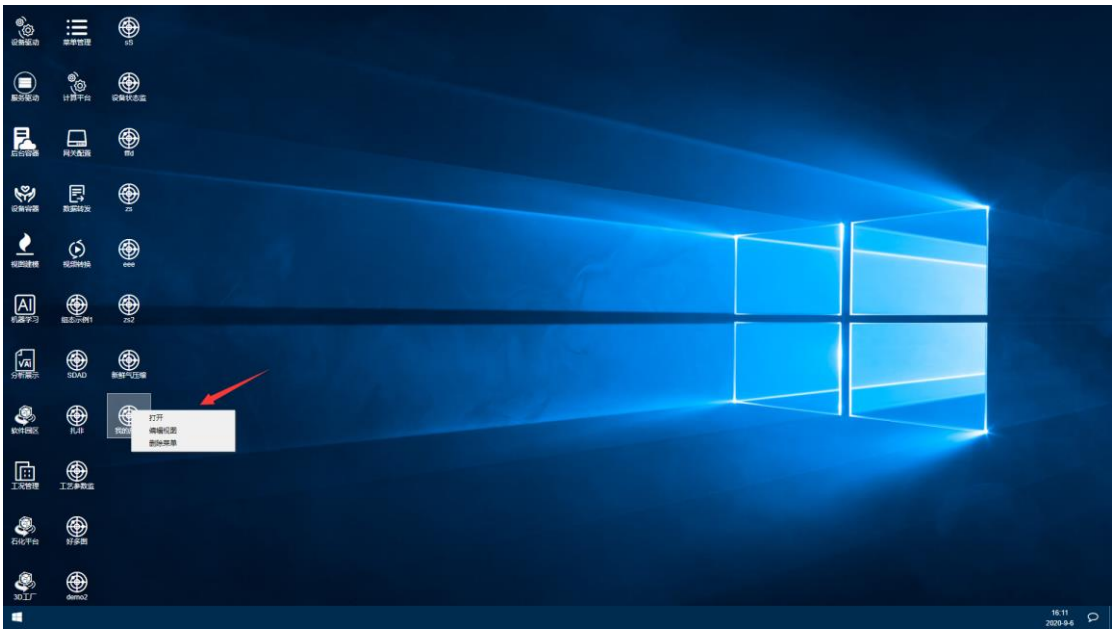


图 保存到桌面

【保存】后，关闭 iNeuView 编辑区域，返回到 iNeuOS 桌面，如下图：



8、打开、编辑和删除应用，应用发布到桌面后，可以单击【右键】对应用进行打开、编辑和删除操作，如下图：



9、使用 iNeuView 开发整个应用，完成。

2.4 3500 多个 SVG 矢量图元

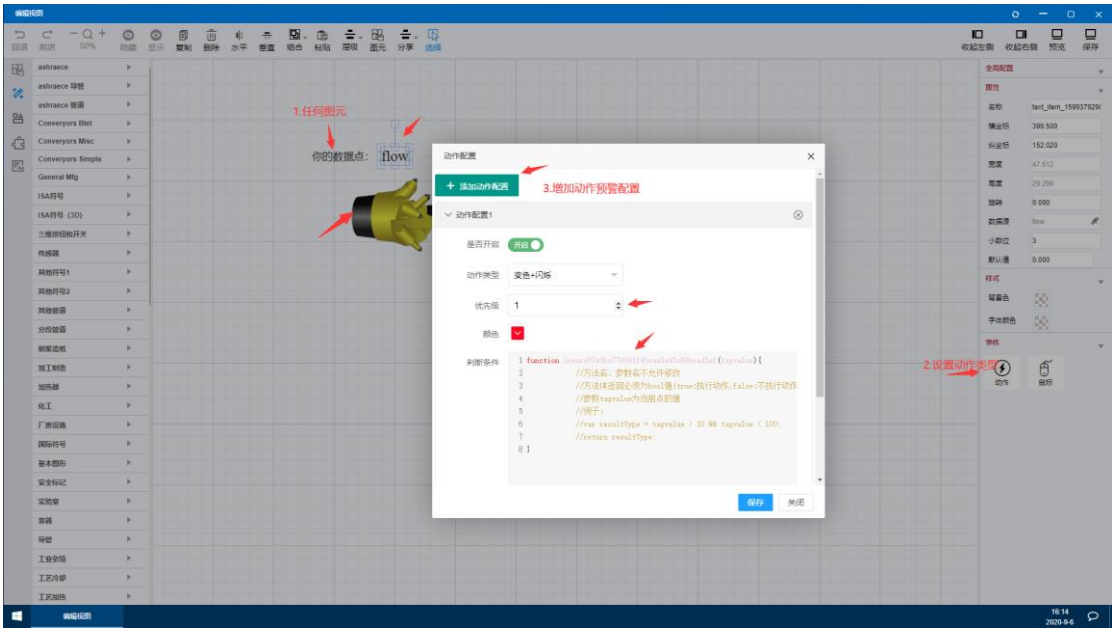
面向钢铁、煤炭、能源、电力、加工制造、环保、楼宇等行业应用，总共有 60 多个图元类别，3500 个图元信息，包括：3D-按钮、交通及工具、人机接口、传感器、分段管、制冷设备、制热设备、加工设备、容器、工业化、工厂

设备、建筑物、建筑设备、控制器、暖通空调、机器、水和废水、泵、流量计、用户上载、电力、电机、空调、简单传送、管道、精加工设备、罐、自然、计算机设备、输送设备、造纸、采暖管道、采矿、锅炉、阀门、面板、风机、食物和其他等。如下图：



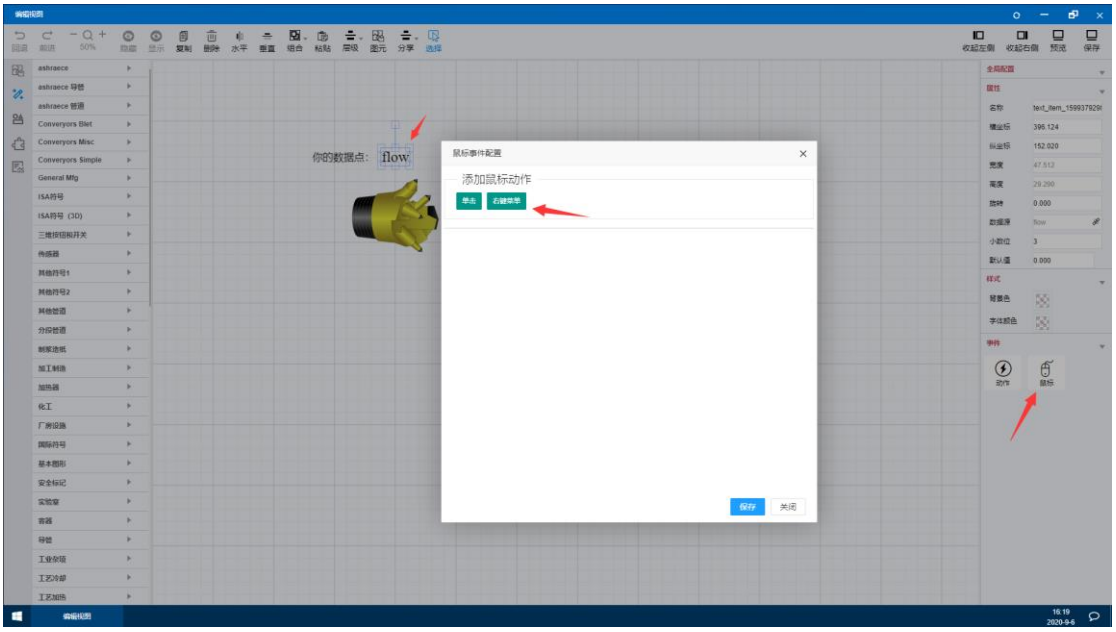
2.5 多方案多级别脚本预警配置

在组态编辑区域上的任何一个图元都可以设置该图元的【动作】,【添加动作配置】可以对该图元设置是否开启、动作类型、优先级、颜色和 JS 脚本判断条件等，支持扩展属性配置。如下图：

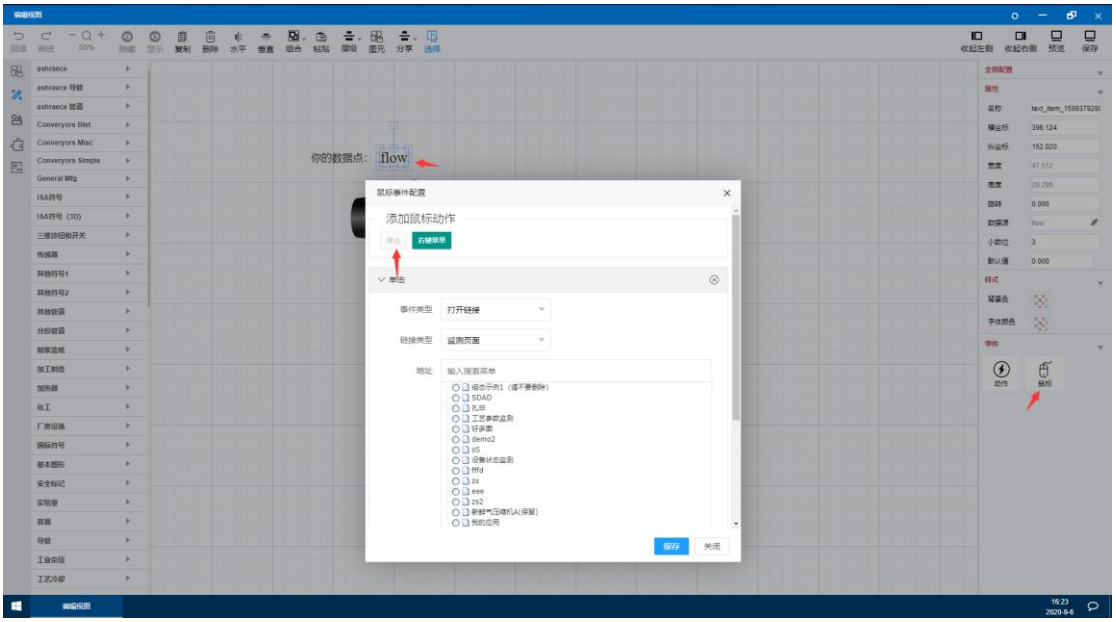


2.6 单击和右键菜单响应事件

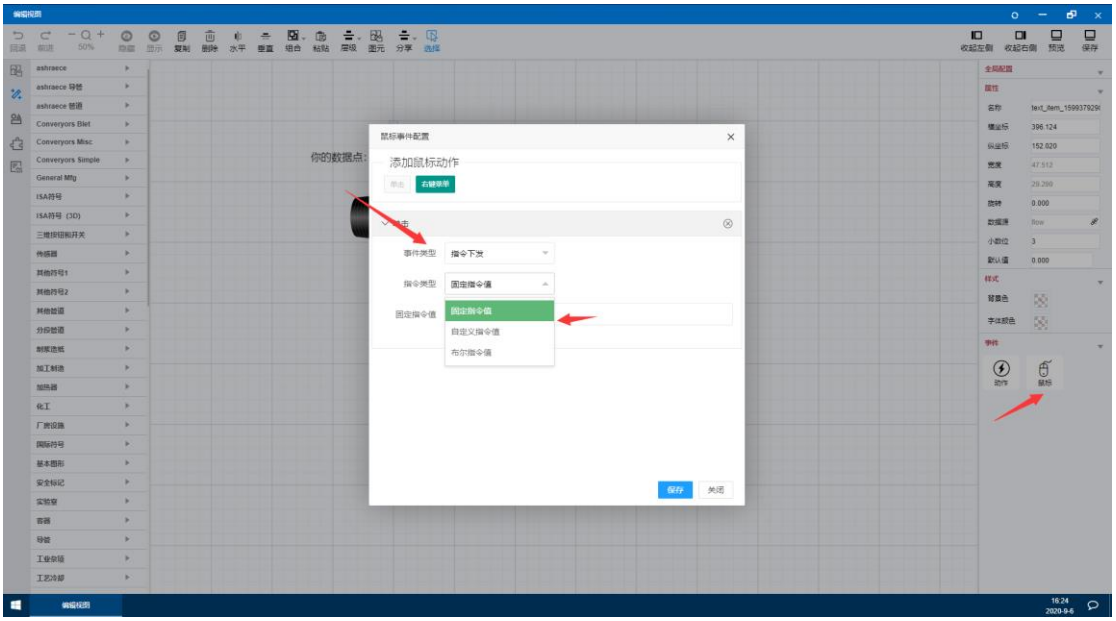
在组态编辑区域上的任何一个图元都可以设置该图元的【鼠标】事件，包括：单击事件和右键菜单事件。可以对事件设置触发行为，例如：下发控制命令和打开内部应用或外部应用。如下图：



打开内部应用或外部应用，如下图：

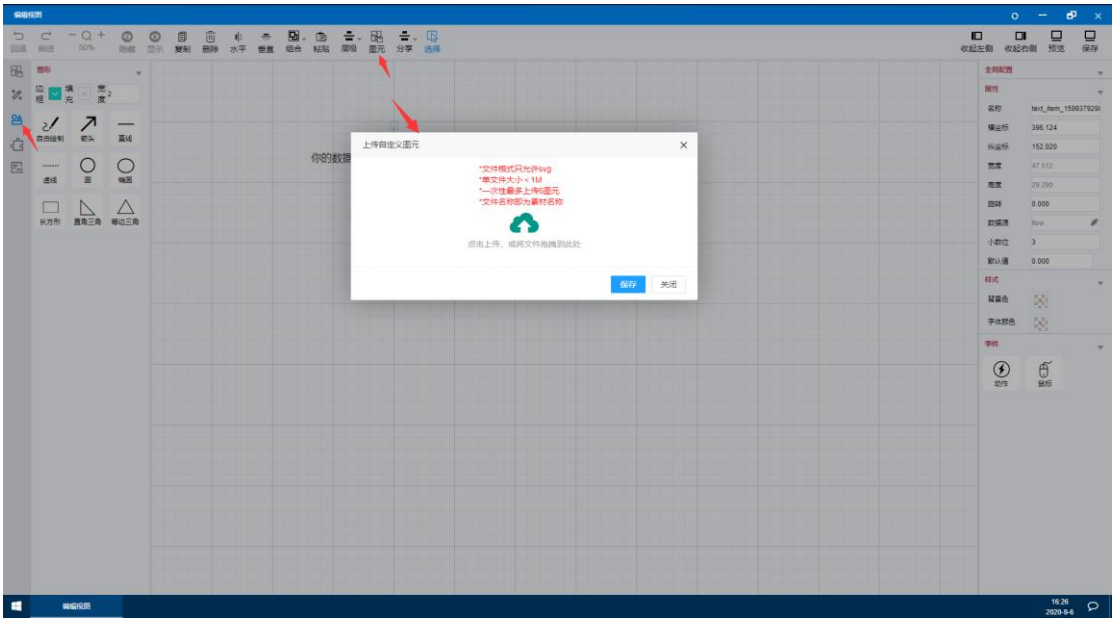


设置下发控制命令，如下图：



2.7 上传图元和自定义绘制图形

选择工具栏【图元】可以上传自定义图元，选择分类栏中上的第三个分类可以自定义绘制图元信息，如下图：

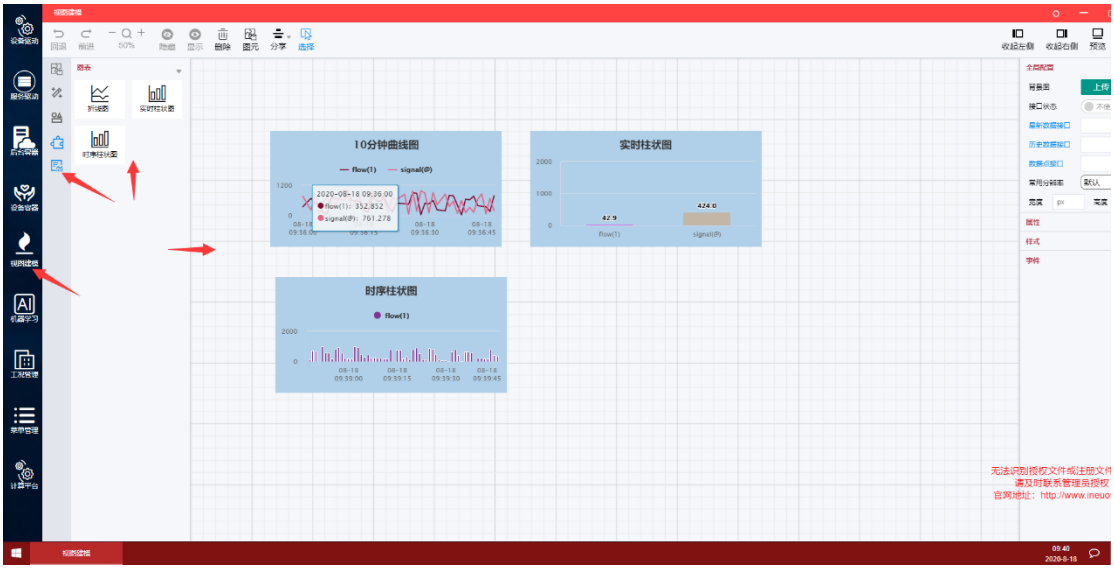


2.8 丰富的图表数据显示组件

选择第 5 个元件库分类，其中包括：拆线图、实时柱状图、时序柱状图和限制预警图。拆线图是通常的曲线图、实时柱状图是显示数据点的当前最新

值、时序柱状图是显示数据点的一段时间范围的数据、限制预警图是根据限制设置参数动态改变实时柱状图颜色。

图形化接口进行了标准化设计，以适配通用或定制化图形报表，满足行业特定应用场景的快速需求。具体的编辑，如下图：



2.9 低延时实时视频接入

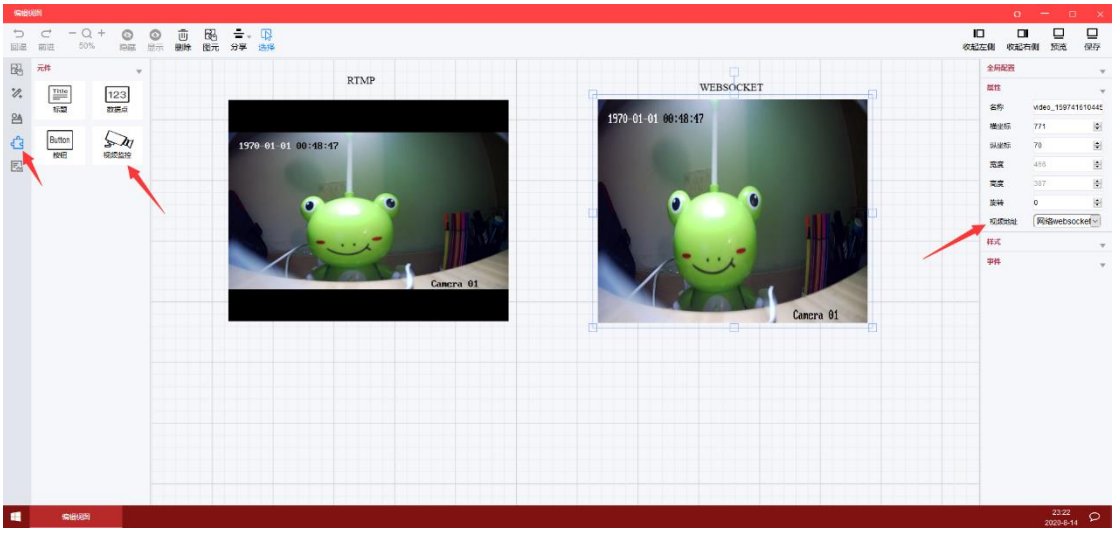
海康和大华等厂家的摄像头对外提供 rtsp 协议，此协议一般无法直接在浏览器上显示实时视频流。

Web 组态（iNeuView）提供了在线实时视频流转换服务，把 rtsp 协议转换成 rtmp 协议或 websocket 协议，满足低延时实时在组态上显示视频流的需求。

WEB 组态（iNeuView）支持实时视频协议转换，满足在浏览器上实时显示视频。视频管理列表，如下图：

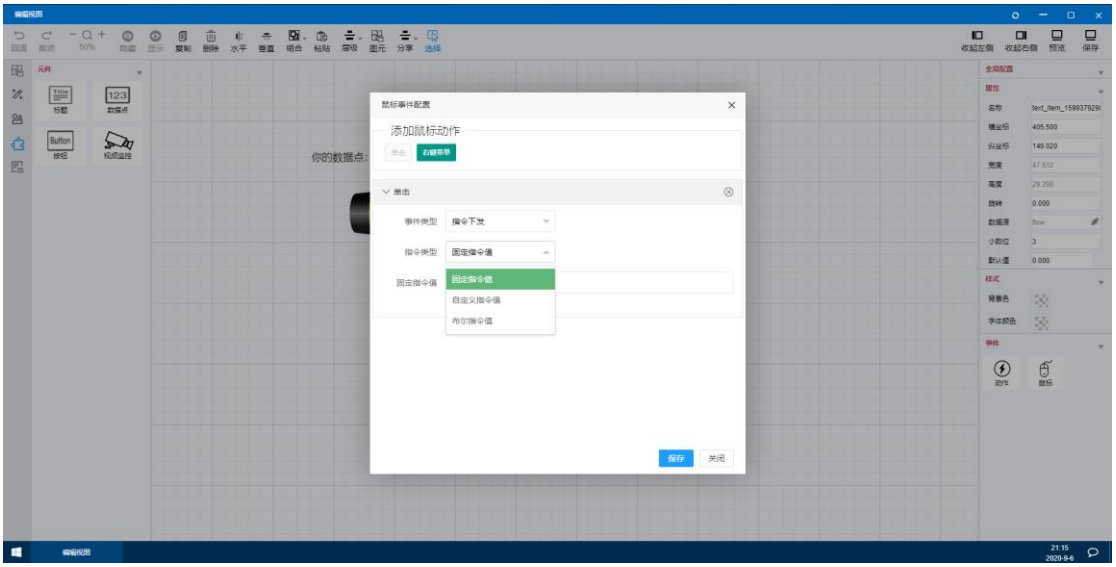


在元件列表中选择【视频监控】元件，拖动到组态画布上，选择右边【视频地址】属性，选择在视频管理列表中的视频名称。如下图：



2.10 控制命令优化级下发

控制端可以理解为 iNeuView 组态，控制操作通过交互协议与后台的 iNeuKernel 进行交互，进一步与设备驱动交互，对设备进行控制操作，当然其他终端的操作链路也是这样的。对于编辑区域上的任何一个图形元件都可以编辑单击事件和右键菜单事件，指令类型包括：固定指令、自定义指令和布尔指令等。如下图：



2.11 开放式的二次开发接口

完全支持二次开发，与其他系统进行集成，例如：MES、EAM 等。二次

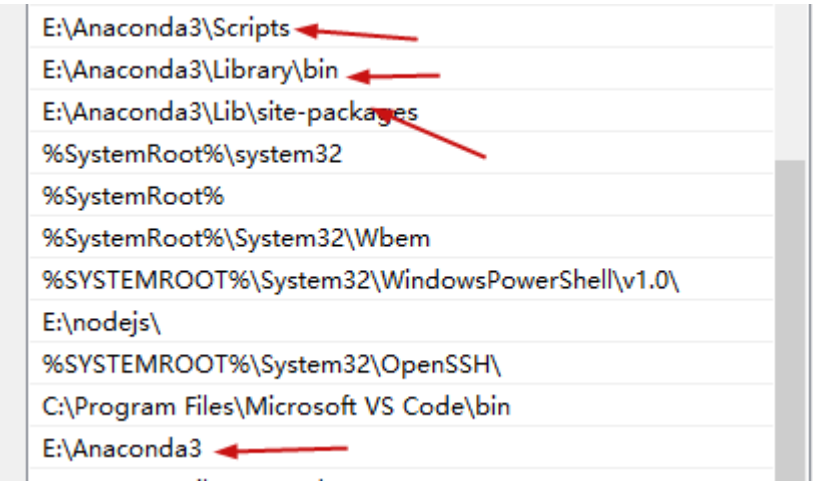
开发参见：《iNeuOS-Token 验证接口.docx》、《iNeuOS-登录跳转.docx》、《iNeuOS-组态菜单集成接口.docx》和《iNeuOS-组态界面数据点扩展接口测试.docx》。

3. 资产模型（机器学习）

3.1 部署配置

算法框架：Anaconda3。版本：Python 3.5.2 |Anaconda 4.1.1 (64-bit)| (default, Jul 5 2016, 11:41:13) [MSC v.1900 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license" for more information。

环境配置：



3.2 操作过程

任务列表页面，查询、添加、编辑、删除任务。

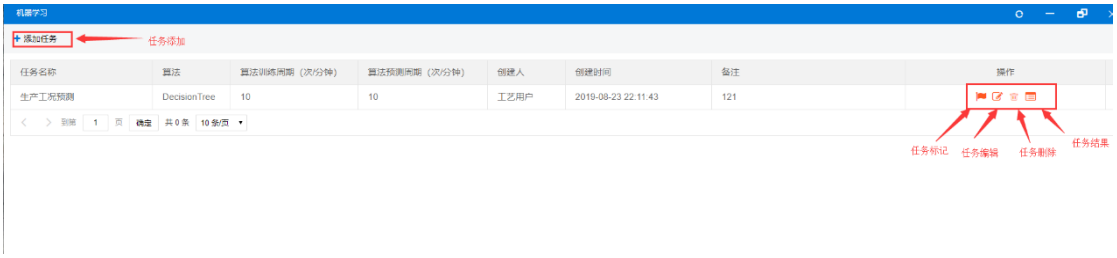


图 1

添加任务。



图 2

属性框点击展示，可输入搜索，勾选点绑定。

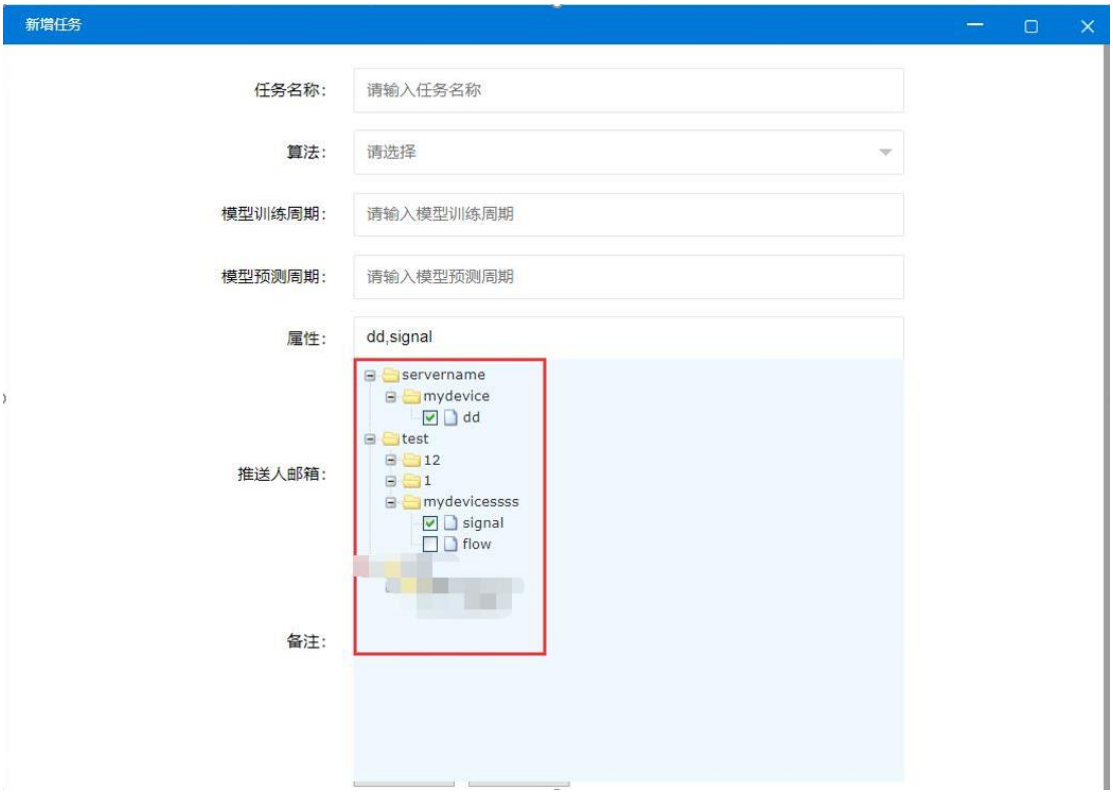


图 3

点击任务编辑，修改任务的基本信息。



图 4

点击任务标记，展开任务列表。

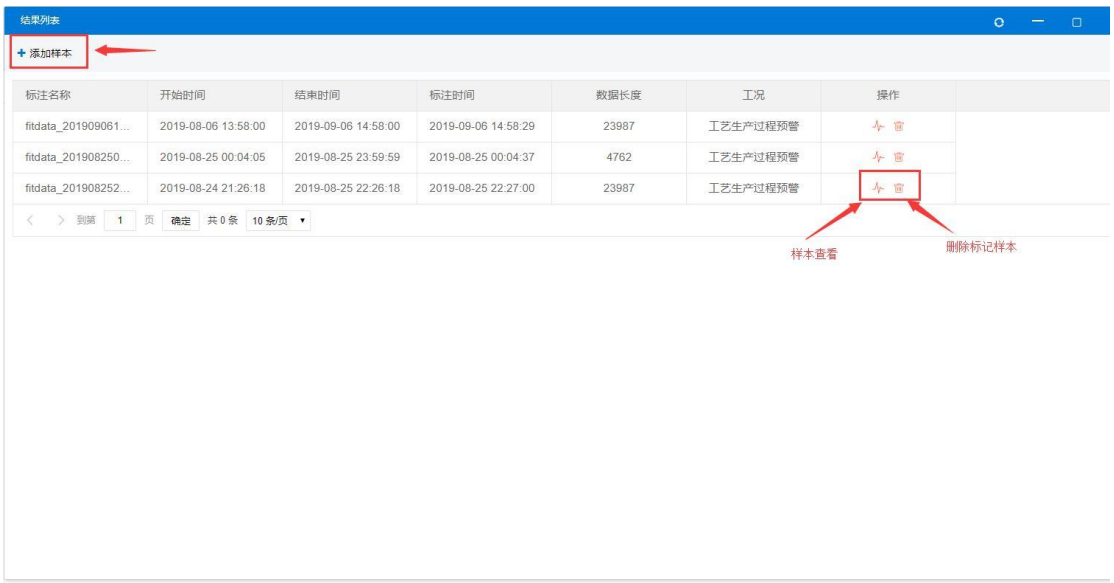


图 5

点击添加样本，选择样本数据。

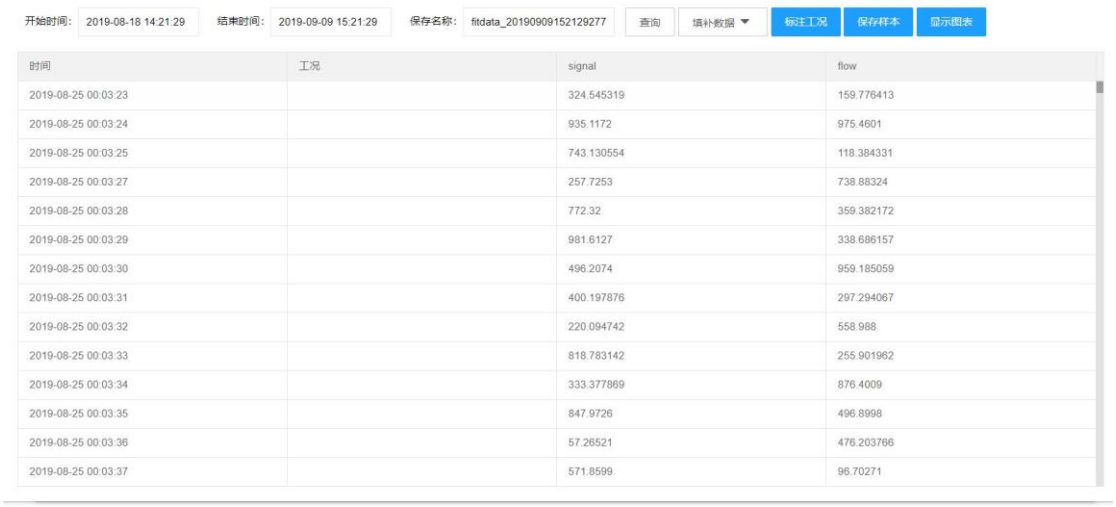


图 6

点击标注工况，对样本数据进行工况标记。

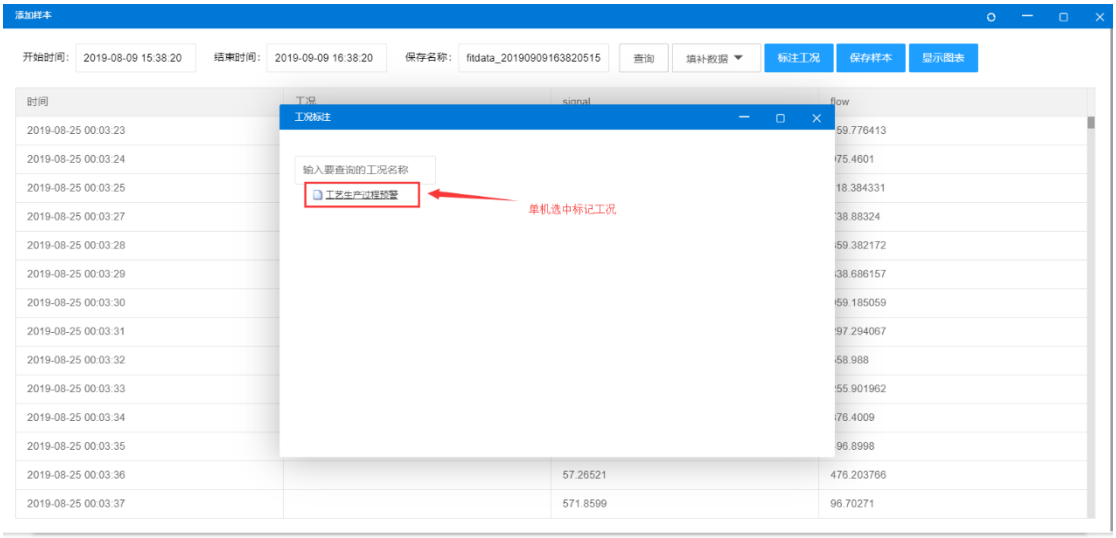


图 7

点击显示图表，以图表的方式展示样本图表。

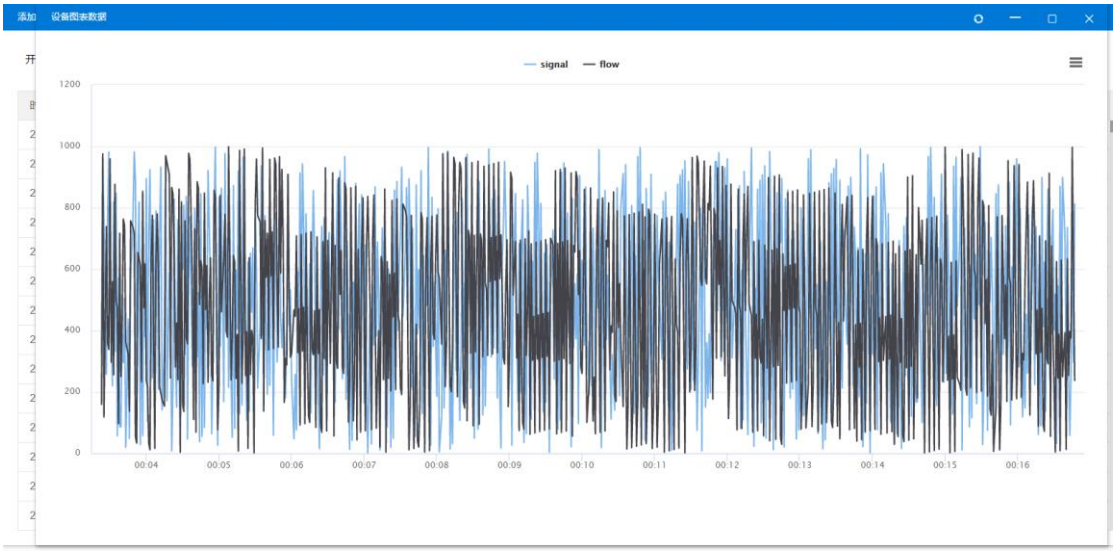


图 8

填补数据，对列表中空数据进行填补数据，支持最大值、最小值和平均值。

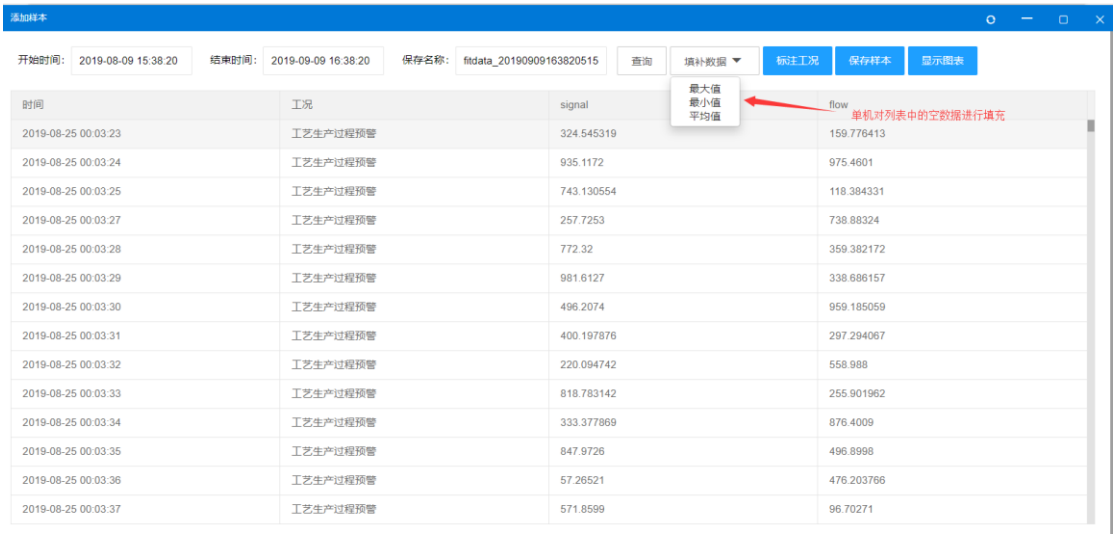


图 9

保存样本，工况标注完成后及数据填充后，输入保存名称（可选），保存样本数据。

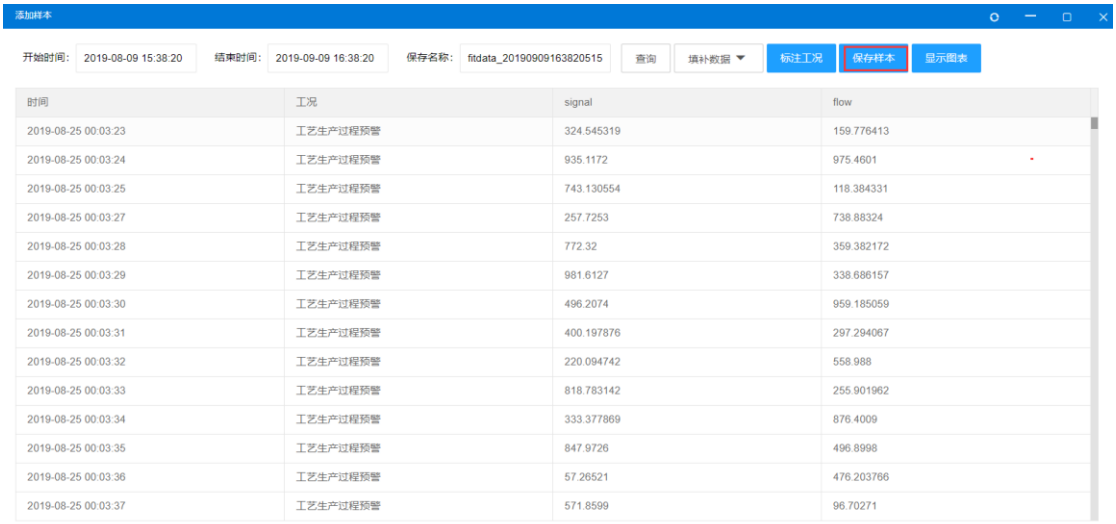


图 10

任务下可多次添加样本数据，机器智能计算预测，删除某标记，其他重新计算

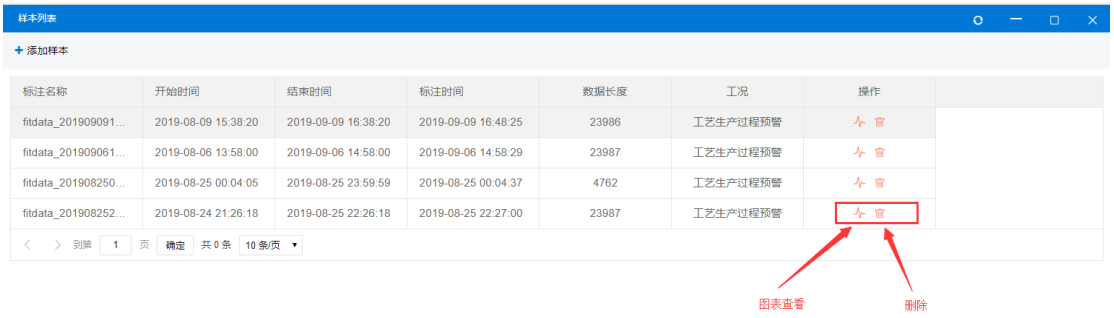


图 11

点击图表查看，以图表的方式查看样本数据。

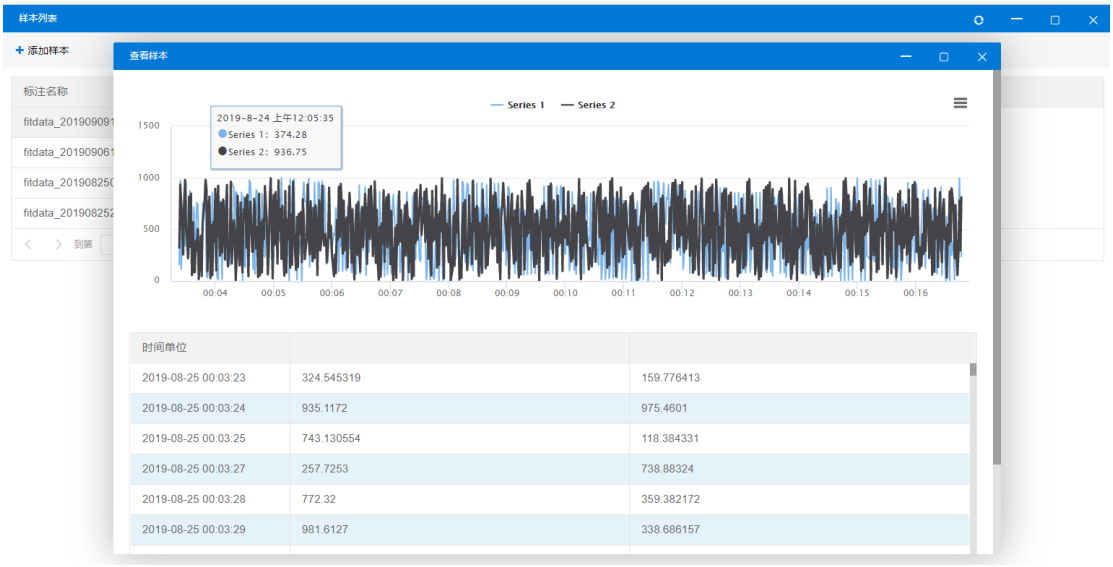


图 12

工况管理，点击可对工况进行添加、修改、删除操作。



图 13

4. 联动控制

4.1 前言

流程控制使用.Net core 编写，完全支持.Net core 语法。基础语法：

https://www.runoob.com/csharp/csharp-basic-syntax.html。流程控制程序外层内置一个大循环，如不需要外层循环需在代码开头执行 stop()。

4.2 内置函数

4.2.1 ctrl

描述：函数可控制所有设备（阀门、温控仪、床等）。

用法：ctrl("设备 ID", "设备名称", "数据点 ID", "数据点名称", “控制值”);

实例：

ctrl("938edf7763bfee92950e4489a5bb625c","3#冷热循环系统", "bc7b65cc56475410aa002fa014efafc11", "1#填充柱目标温度", “160”);

4.2.2 save

描述：函数可保存数值到数据库中（一般用于虚拟数据点保存使用）。

用法：save(“数据点 ID”, "数据点名称", "值");

实例：

save(“9a6efd1e9b4d4a75b06a84910e51029f”, "HS04_HGB01 当前的容量", "0");

4.2.3 getvalue

描述：函数可获取数据点最新数据值。支持两种用法：

用法：

getvalue("设备 ID", "设备名称", "数据点 ID", "数据点名称");

getvalue("数据点 ID", "数据点名称");

实例：

getvalue("6aa49827b912ad2a83642f475a9bf727","2#手套箱 VI 模块-8","3162c1fb6b80a4261a52eadb19a0746c0","HS04-PI001")

getvalue(“b65f6aec35b674eb89f3069ed9211bf76”,"HS04_HGB01 当前的容

量标识");

4.2.4 getpara

描述：函数可获取根据变量名称获取运行参数设定值。

用法：getpara("运行参数变量名称");

实例：getpara("HS04-CV01-CV");

4.2.5 delay

描述：程序停止多少毫秒后继续执行，参数为毫秒。

用法：delay(1000);//1 秒钟

实例：

delay(1000);

4.2.6 log

描述：函数将内容原样输出到日志中。

用法：log(“内容”);

实例：log(“内容”);

4.2.7 start

描述：程序启动。

用法：start();

实例：start();

4.2.8 stop

描述：停止流程控制外层循环。

用法：stop();

实例：stop();

4.2.9 exit

描述：当前流程控制退出。

用法：exit();

实例：exit();

4.3 自定义封装函数

4.3.1 tryParseDouble

函数：

```
/// <summary>
///  字符串转数字
/// </summary>
/// <param name="val"></param>
/// <returns></returns>
private double tryParseDouble(string val)
{
    double tempVal = 0;
    if (!string.IsNullOrWhiteSpace(val))
    {
        double.TryParse(val, out tempVal);
    }
    return tempVal;
}
```

描述：

将字符串转换成 double 类型，一般用于字符串需运算时进行转换。

用法：

double val = tryParseDouble(“这里是一个字符串”);

实例：

double HS04_PI001_1= tryParseDouble(getvalue("6aa49827b912ad2a83642f475a9bf727","2#手套箱 VI 模块-8","3162c1fb6b80a4261a52eadb19a0746c0","HS04-PI001"));

4.3.2 awaitCtrl

函数：

```
/// <summary>
///  下发控制命令，直到命令下发完成后才返回
/// </summary>
/// <param name="deviceId">设备 ID</param>
/// <param name="deviceName">设备名称</param>
/// <param name="tagId">数据点 ID</param>
/// <param name="tagName">数据点名称</param>
/// <param name="value">控制命令值</param>
public void awaitCtrl(string deviceId,string deviceName,string tagId,string tagName,string value)
{
    string tempValue    = getvalue(deviceId, deviceName, tagId, tagName);
    if(value != tempValue)
    {
        //下发命令
        ctrl( deviceId,  deviceName,  tagId,  tagName,  value);
        while (true)
        {
            tempValue = getvalue(deviceId, deviceName, tagId, tagName);
            //设备命令操作完成后才返回
            if(value == tempValue)
            {
                break;
            }
            delay(500);
```

```
            }
        }
    }
```

描述：

下发控制命令，直到命令下发完成后才返回。只用于阀门控制，阀门状态切换成功后执行后续操作。

用法：

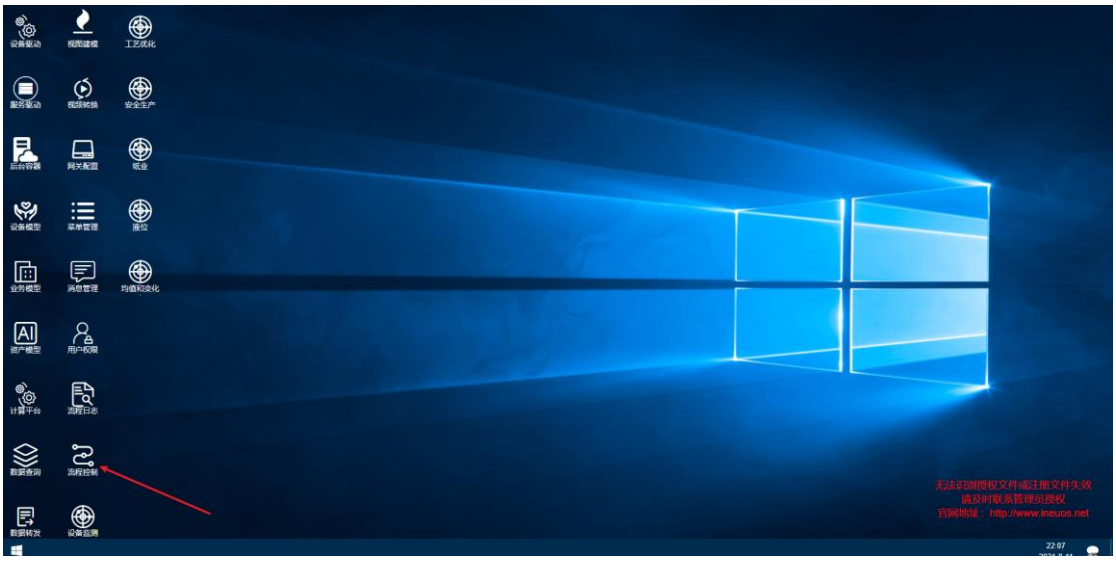
awaitCtrl("设备 ID", "设备名称", "数据点 ID", "数据点名称", “控制值”);

实例：

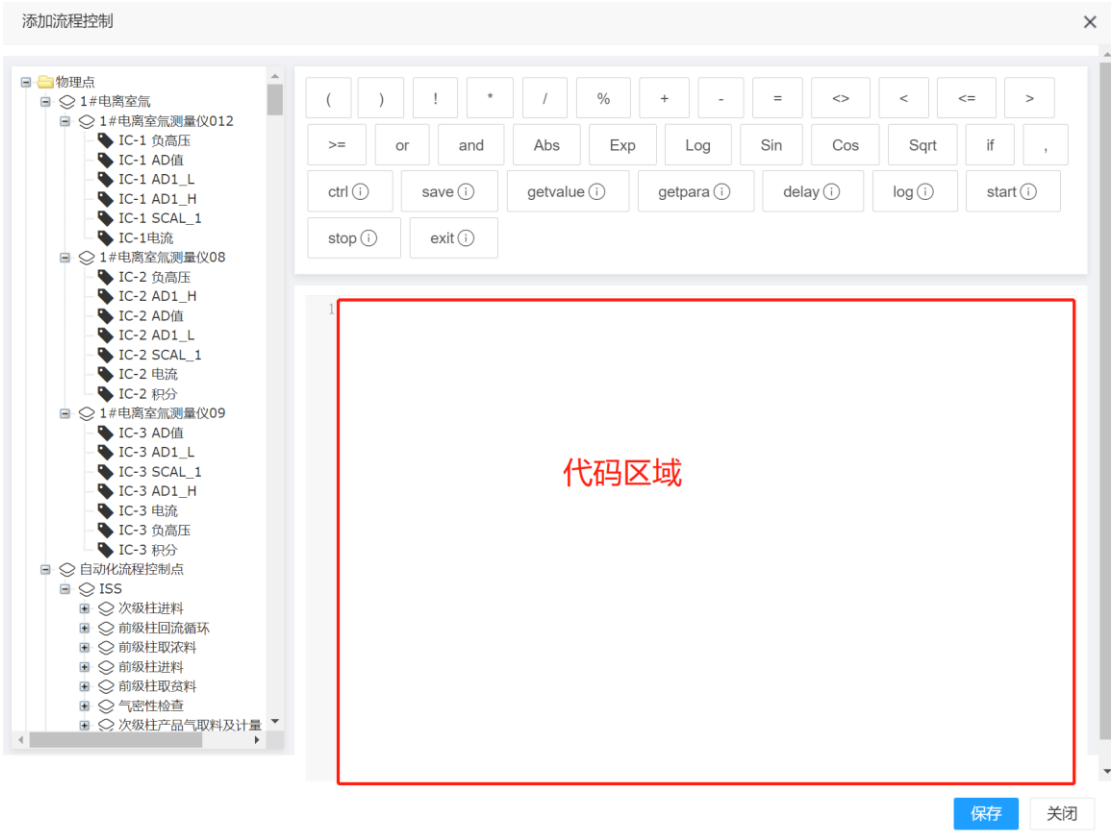
awaitCtrl("db23c44c3abf8354afab230c02189b9f", "2#手套箱 DO 模块 9-控制电磁阀", "b71c51dd8c27a4769b7344485aa549b0f", "HS04 GV002", "1");

4.4 使用介绍

浏览器访问 iNeuOS 网页，双击打开“流程控制”功能。



打开功能页面后，点击“添加”按钮，弹出添加流程控制界面，在代码区域键入流程控制代码。



代码编辑完成后，点击保存完成添加。（间隔时长为外层循环的循环间隔时间）

